

A lossless and fully parallel spintronic compute-in-memory macro for artificial intelligence chips

Received: 13 January 2025

Accepted: 16 September 2025

Published online: 16 October 2025

 Check for updates

Humiao Li^{1,6}, Zheng Chai^{1,2,3,6}, Weirong Dong¹, Junjie He¹, Ruijie Peng¹, Shiheng Li¹, Zhen Kong¹, Xihui Yuan², Xianwang Wang², Zhengke Yang¹, Haoran Lyu¹, Haofeng Yu¹, Xue Zhou², Jiamin Li¹, Feichi Zhou¹, Yida Li¹, Zongben Xu^{3,4}, Tai Min^{2,3,5}✉ & Longyang Lin¹✉

Non-volatile compute-in-memory macros can reduce data transfer between processing and memory units, providing fast and energy-efficient artificial intelligence computations. However, the non-volatile compute-in-memory architecture typically relies on analogue computing, which is limited in terms of accuracy, scalability and robustness. Here we report a 64-kb non-volatile digital compute-in-memory macro based on 40-nm spin-transfer torque magnetic random-access memory technology. Our macro features in situ multiplication and digitization at the bitcell level, precision-reconfigurable digital addition and accumulation at the macro level and a toggle-rate-aware training scheme at the algorithm level. The macro supports lossless matrix–vector multiplications with flexible input and weight precisions (4, 8, 12 and 16 bits), and can achieve a software-equivalent inference accuracy for a residual network at 8-bit precision and physics-informed neural networks at 16-bit precision. Our non-volatile compute-in-memory macro has computation latencies of 7.4–29.6 ns and energy efficiencies of 7.02–112.3 tera-operations per second per watt for fully parallel matrix–vector multiplications across precision configurations ranging from 4 to 16 bits.

Artificial intelligence (AI) systems require energy-efficient computation at rapid response times^{1,2}. However, deploying neural networks (NNs) on traditional von Neumann architectures creates performance bottlenecks in terms of energy and time costs due to data movement between the processing and memory units (the memory wall)^{2–4}. Non-volatile compute-in-memory (nvCIM) architectures can avoid such data movement, providing AI processing directly within non-volatile memory such as ferroelectric random-access memory^{5–7}, resistive

random-access memory (RRAM)^{8–16}, magnetoresistive random-access memory (MRAM)^{17–20} and phase-change memory (PCM)^{21–25} via analogue computation.

Analogue nvCIM architectures are energy efficient and can have a compact cell area footprint with high memory density. However, their analogue nature introduces intrinsic challenges—including limited bit precision, nonlinearity, process–voltage–temperature variations and quantization noise—that degrade computation accuracy and

¹School of Microelectronics, Southern University of Science and Technology, Shenzhen, China. ²State Key Laboratory for Mechanical Behavior of Materials, and School of Materials Science and Engineering, Xi'an Jiaotong University, Xi'an, China. ³Pazhou Laboratory (Huangpu) and Peng Cheng Laboratory, Guangzhou, China. ⁴Ministry of Education Key Laboratory of Intelligent Networks and Network Security, and School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an, China. ⁵National Laboratory of Solid-State Microstructures, and School of Materials Science and Intelligent Engineering, Nanjing University, Suzhou, China. ⁶These authors contributed equally: Humiao Li, Zheng Chai. ✉e-mail: tai.min@nju.edu.cn; linly@sustech.edu.cn

robustness^{26–28}. In addition, analogue nvCIM relies on components such as digital-to-analogue converters and analogue-to-digital converters for data conversion, limiting scalability in advanced technology nodes due to reduced voltage margins and precision, along with increased design complexity, area and energy overhead^{28–30}. At the same time, AI models, such as AlphaFold and physics-informed neural networks (PINNs), demand accurate, high-precision, and reliable matrix–vector multiplications (MVMs) for solving complex problems^{31,32}, which is challenging for analogue nvCIM.

Conventional digital compute-in-memory (DCIM) macros^{33–38} perform fully digital operations within the volatile digital memory such as static random-access memory, and can provide a lossless and high-precision MVM, in which the computations are accurate and free from information loss during the CIM process. In addition, the almost-unlimited endurance of volatile digital memory supports the long operational lifespan required by AI systems^{39,40}. However, the volatility of digital memory creates drawbacks, including high initialization energy and latency from data reloading at startup, as well as considerable leakage energy from the continuous power supply required for data retention^{39,40}. All of these degrade the energy efficiency of such systems.

In this Article, we report a 64-kb nvDCIM macro based on foundry spin-transfer torque magnetic random-access memory (STT-MRAM) technology using a 40-nm complementary metal–oxide–semiconductor (CMOS) process (Fig. 1a,b). The macro has a bitcell architecture that enables direct 1-bit in-bitcell multiplication and digitization (IBMD), producing a digital output equivalent to the AND logic operation between the digital input and the analogue non-volatile weight. Lossless MVMs and high parallelism are achieved through full-precision adder trees, enabling fully parallel MVMs with low latency and high throughput. In addition, a reconfigurable accumulator supports multiple bit precisions (4, 8, 12 and 16 bits), providing flexibility for diverse AI workloads. We use an algorithm-level toggle-rate-aware training method to improve energy efficiency by up to 109% for residual network (ResNet)-20, with accuracy degradation of only 0.68%. Our nvDCIM macro achieves inference accuracy identical to the software baseline for ResNet-20 at 8-bit precision and PINNs at 16-bit precision. Our macro delivers computation latencies of 7.4–29.6 ns and energy efficiencies of 7.02–112.3 tera-operations per second per watt (TOPS W^{−1}) for fully parallel MVMs at a precision level ranging from 4 to 16 bits.

IBMD scheme

The STT-MRAM is based on a current-controlled magnetic tunnel junction (MTJ) structure consisting of two ferromagnetic layers separated by a thin insulating barrier. This structure is integrated within the CMOS technology, situated between two metal layers (Metal-6 and Metal-7). One of the ferromagnetic layers serves as the reference layer (RL) with fixed magnetization, whereas the other acts as the free layer, whose magnetization can switch between two states: parallel (P) or antiparallel (AP) states relative to the RL. These states correspond to distinct resistance levels—low resistance (R_L) in the P configuration and high resistance (R_H) in the AP configuration. The two resistance states represent binary data, with R_L encoding ‘0’ and R_H encoding ‘1’ (further details about the STT-MRAM device structure, electrical characteristics and reliability analysis are provided in Extended Data Fig. 1 and Methods).

Figure 2a shows a schematic of our IBMD bitcell. This bitcell integrates two one-transistor–one-MTJ (1T1MTJ) structures with a latch, sharing the tail p-type field-effect transistor (M9) and n-type field-effect transistor (M10) with other bitcells in the same row. The left MTJ (denoted as R0) stores the weight (W), whereas the right MTJ (R1) holds its complementary value ($\bar{W}$), enabling a comparison operation critical for IBMD functionality. The bitcell supports standard memory operations, including write and read, controlled through the source line (SL), bit line (BL) and complementary bit line (BLB).

To facilitate efficient writing in the IBMD bitcell, we used a two-step write scheme, eliminating the need for additional transistors (Fig. 2b). In this scheme, the gates of the selection transistors (M5 and M6) are grounded. During Step 1, a 0 is written to both MTJs by applying the write voltage (V_W) to the SL of the selected row through a transmission gate, and grounding both BL and BLB. This drives currents through both MTJs, setting them to the P state. During Step 2, a 1 is selectively written to the targeted MTJ. SL is grounded in this step, whereas the BL and BLB voltages alternate between V_W and ground, depending on the desired weight to be stored. By the end of the two-step process, the MTJs store complementary values, accurately encoding the weight data. This scheme ensures an efficient preparation of the bitcell for subsequent read and compute operations and maintain a minimal circuit design.

The input signal IN and its complementary counterpart INB enable input-triggered digitization (Fig. 2c shows the operation waveform). During the read and compute operations, BL and BLB are connected to the supply voltage (V_{DD}). When IN is pulled to ground (IN = 0), transistors M5 and M6 precharge the Di+ and Di− nodes to V_{DD} , which in turn causes M8 to discharge the OUT node to ground. As IN transitions to V_{DD} (IN = 1), the tail transistors (M9 and M10) turn on, initiating the latch stage. Simultaneously, the parasitic capacitors at the Di+ and Di− nodes start discharging through the MTJs (R0 and R1, respectively), initiating the input-triggered sensing and digitization process. The voltage decay across the capacitors follows the exponential relationship:

$$V = V_0 e^{-\frac{t}{RC}}, \quad (1)$$

where the decay rate depends on the resistance (R) and capacitance (C). In the IBMD bitcell, identical parasitic capacitances at Di+ and Di− ensure that the discharge rate is determined by the resistance of the MTJs. The resistance difference between R0 and R1 generates a weight-dependent differential voltage ΔV_{Di} developed between Di+ and Di−. This differential voltage ΔV_{Di} is then propagated to the cross-coupled inverters in the latch stage via an intermediate stage, formed by transistors M7 and M8. The latch stage amplifies the differential signals, driving the OUT node to V_{DD} or ground. The digital output represents the AND logic operation between the digital input (IN) and the analogue non-volatile weight (W) stored in the MTJs. When IN = 0, the OUT node and its counterpart are pulled down to GND via M8 and M7, respectively, ensuring stable operation. However, when IN = 1, potential read errors may arise due to load imbalance, as the OUT node is connected to the adder tree, whereas its counterpart remains unconnected. To address this issue, the right half of the latch stage (M3 and M4) is designed slightly stronger than the left half (M1 and M2). This is achieved by shortening the channel lengths of the right inverter and maintain identical channel widths, ensuring reliable read and compute operations.

Post-layout simulations at the typical–typical process corner under a nominal voltage demonstrate the high-speed operation of the IBMD bitcell, achieving an output rise delay of 88 ps and a fall delay of 22 ps. Crucially, no further digitization occurs if IN remains constant across subsequent clock cycles (for example, IN = 11...1 or 00...0). In this case, no precharge is required, and the OUT state is retained, resulting in zero dynamic power consumption. Figure 2d shows the operation table supporting array-wise write, read and compute functionalities.

Conventional analogue nvCIM designs typically feature a compact cell area, making them well suited for high-density memory arrays. However, the analogue nature of the computing process inherently limits computational accuracy and robustness. By contrast, our IBMD bitcell enables digital AND logic operations between the digital input and non-volatile weight, providing robust and accurate computation in the digital domain. Although the cell area is larger than that of a conventional analogue nvCIM (Extended Data Table 1), this trade-off delivers enhanced computational accuracy and robustness, addressing the key limitations of analogue nvCIM architectures.

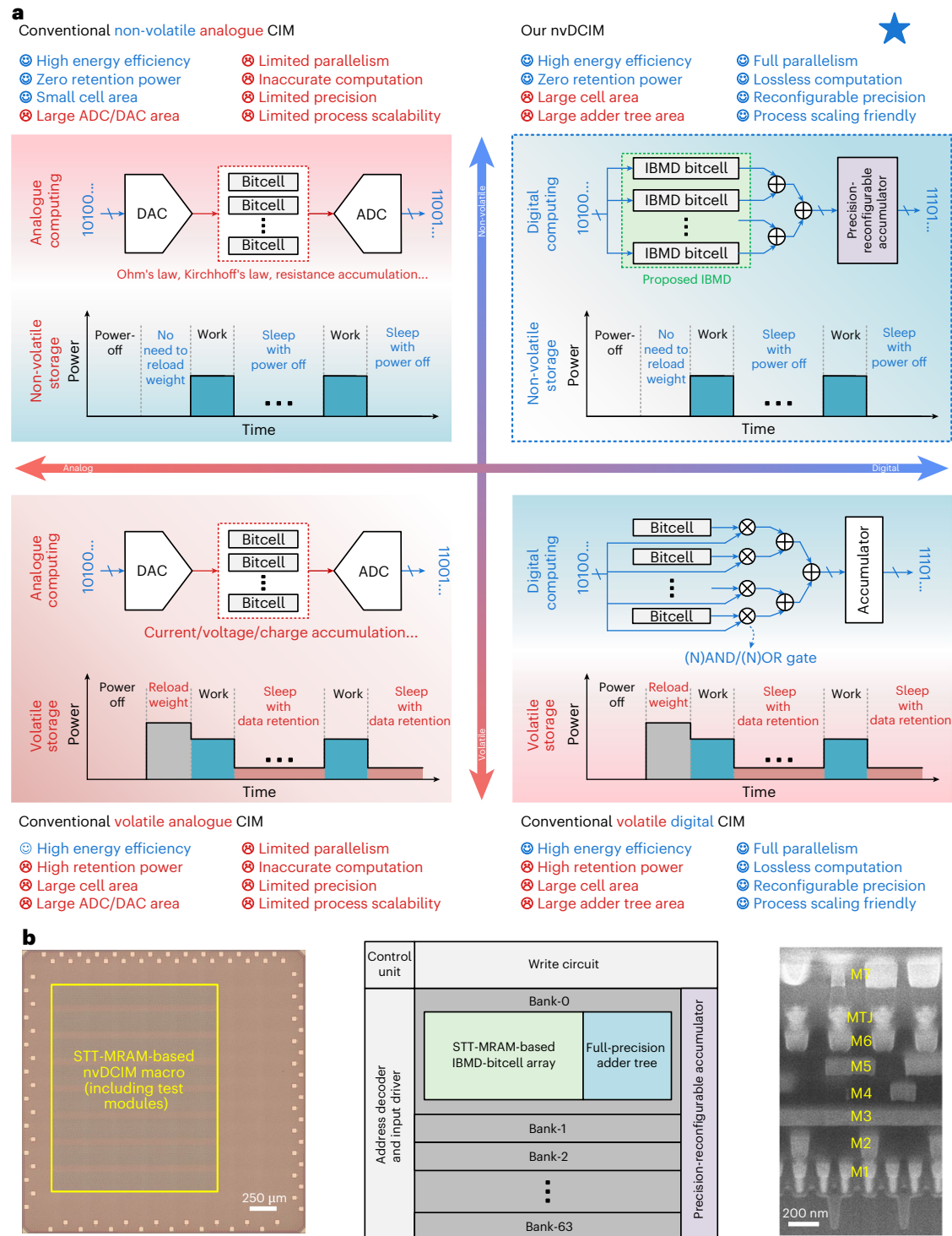


Fig. 1 | Motivation and overview of the nvDCIM macro. a, Comparison of conventional compute-in-memory (CIM) architectures with our nvDCIM. Conventional analogue nvCIM^{5–25} utilizes analogue computing principles (for example, Ohm's law and Kirchhoff's law), suffering from low precision, limited parallelism, inaccurate computation and poor scalability due to its reliance on digital-to-analogue converters (DACs) and analogue-to-digital converters (ADCs). Conventional volatile analogue CIM^{57–59} faces all these challenges and additionally requires high retention power. Although conventional

volatile digital CIM^{33–38} overcomes analogue limitations and achieves lossless computation, it suffers from high power requirements for retention and startup. By contrast, our nvDCIM leverages the IBMD scheme, digital adder tree and accumulator to achieve lossless computation, reconfigurable precision, full parallelism and excellent process scalability, as well as maintaining high energy efficiency and zero retention power. **b**, Die photograph (left), macro structure (middle) and transmission electron microscopy (TEM) image of the fabricated die, with the STT-MRAM located between Metal-6 (M6) and Metal-7 (M7).

The read and compute yields of the IBMD bitcell are validated through a combination of measurements and post-layout Monte Carlo simulations across a wide range of V_{DD} values (Fig. 2e). In the Monte Carlo simulation, the read yield evaluation focused on two worst-case

read scenarios, specifically ($\text{IN} = 1, W = 0$) and ($\text{IN} = 1, W = 1$), which are simulated consecutively under the same process variation point. A pass is defined as having no read error in both read operations. At a supply voltage of 0.65 V, simulations indicate a yield of 3.02 standard

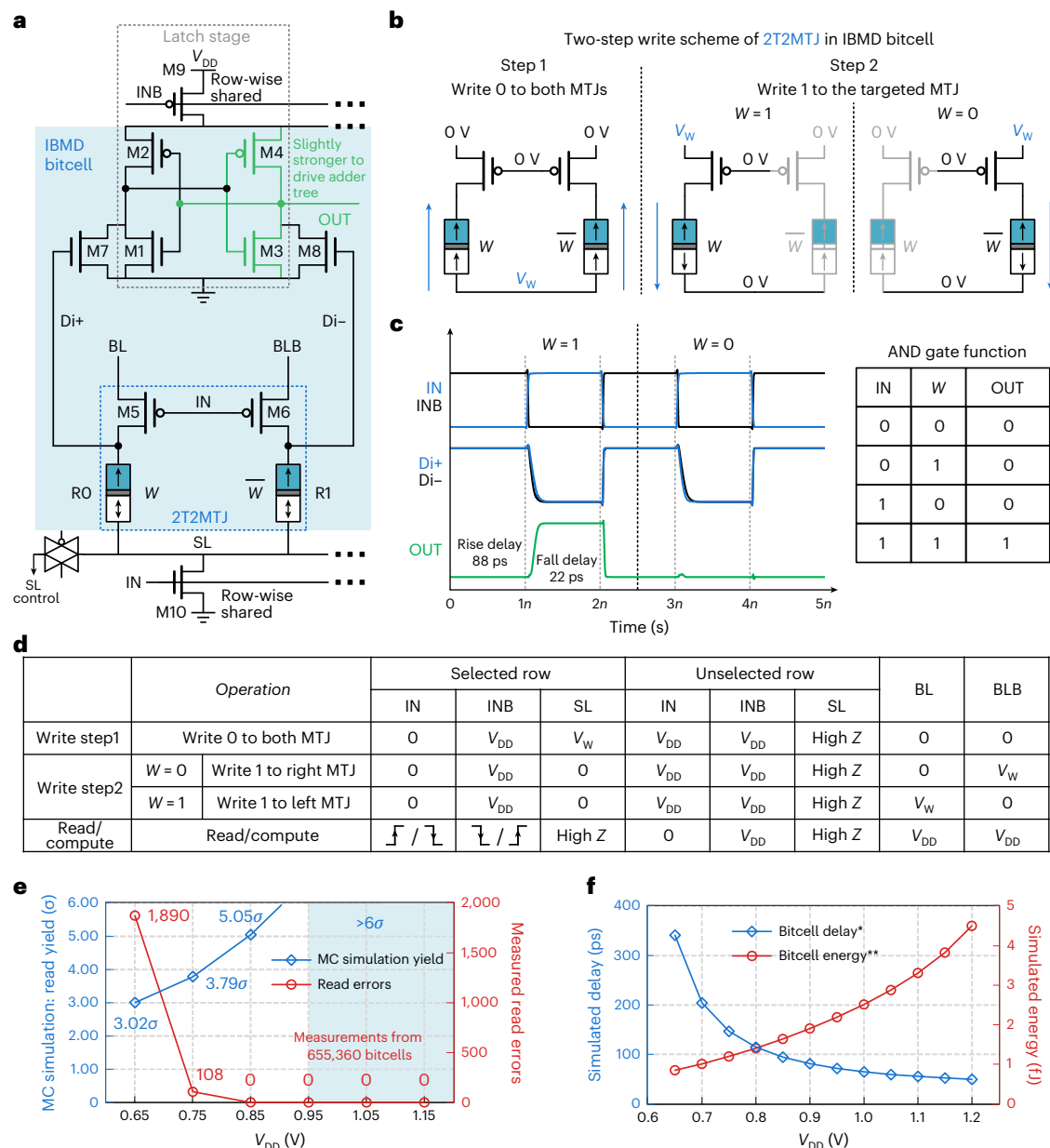


Fig. 2 | Overview of the IBMD bitcell. a, Schematic of the IBMD bitcell featuring a two-stage structure: a 2T2MTJ stage for analogue non-volatile weight storage and a latching stage, with two row-wise shared transistors. **b**, Two-step write scheme for programming complementary data to the MTJs. **c**, Operation waveforms and timing diagram for the read and compute schemes, achieving a rise delay of 88 ps and a fall delay of 22 ps (post-layout simulation under nominal voltage of 1.1 V in the typical–typical corner), demonstrating the AND gate functionality with the

corresponding truth table. **d**, Operation table supporting array-wise write, read and compute functionalities. **e**, Measured read errors (red, right axis) and Monte Carlo (MC) simulation results for read yield (blue, left axis) across a V_{DD} range of 0.65–1.15 V. When V_{DD} exceeds 0.95 V (blue-shaded region), the simulated read yield surpasses 6σ . **f**, Simulated IBMD-bitcell read delay and energy consumption across a V_{DD} range of 0.65–1.20 V. *Bitcell delay = $(\text{delay}_{\text{rise}} + \text{delay}_{\text{fall}})/2$. **Average energy per toggle.

deviations (σ) and measurements across 655,360 bitcells reveal 1,890 read errors. As V_{DD} increases to 0.85 V or higher, the yield improves to over 5.05σ in simulations, with no errors observed in the measurements. The simulated bitcell read/compute delay and energy consumption range from 49.5 ps to 341 ps and 0.85 fJ bit⁻¹ to 4.5 fJ bit⁻¹, respectively, as V_{DD} increases from 0.65 V to 1.2 V (Fig. 2f).

nvDCIM macro

Recent advancements in hybrid spintronic–CMOS architectures and approximate computing techniques^{41–48} have demonstrated improvements in energy efficiency, throughput and security. However, many hybrid spintronic–CMOS designs face limitations in supporting fully parallel computation and precision reconfigurability, which hinder their

scalability and adaptability to diverse AI workloads. Similarly, although approximate computing techniques enhance energy efficiency, this improvement comes at the cost of accuracy, making them unsuitable for precision-critical applications, such as PINNs (a detailed comparison is provided in Supplementary Table 1 and Supplementary Note 1). By contrast, our nvDCIM macro addresses these limitations by leveraging the IBMD bitcell to perform computations in the digital domain. This design enables fully parallel, lossless and precision-reconfigurable MVM operations with high energy efficiency. These unique features overcome key constraints in existing hybrid spintronic–CMOS architectures, providing a scalable and robust solution for advanced AI applications.

The system architecture of our nvDCIM macro (Fig. 3a) consists of four primary components: (1) a write circuit, which includes an SL

control circuit and a BL/BLB control circuit, specifically designed to enable efficient IBMD-bitcell writing; (2) a control circuit responsible for managing write, read and compute operations; (3) a row circuit with an address decoder for row selection and an input driver for transmitting input signals to the IBMD bitcell; and (4) 64 nvDCIM banks, each containing a $256\text{-row} \times 4\text{-column}$ IBMD-bitcell array, a seven-stage full-precision adder tree and a precision-reconfigurable accumulator that supports flexible input and weight precisions.

In the write mode, the address decoder generates one-hot codes for IN and INB signals, which serve as word lines (WLs) to facilitate row selection. Simultaneously, the SL, BL and BLB control circuits adjust the corresponding voltages to write data into the MTJs in the IBMD bitcells. The two-step write scheme is used to store non-volatile weights in the MTJs. For the selected row, its IN signal is grounded, activating the row's selection p-type field-effect transistors and enabling the write operation. The SL, BL and BLB voltages alternate between V_W and ground, depending on the write step and target data. By contrast, for the unselected rows, their IN signals are set to V_{DD} , ensuring their selection p-type field-effect transistors remain off and isolating their MTJs from voltage switching on BL and BLB. Meanwhile, the SLs of the unselected rows are placed in a high-impedance (high- Z) state by turning off the transmission gates in the SL control circuit, effectively isolating unselected rows and preventing any write-disturb issues.

In the read mode, the address decoder activates the selected row by applying step signals to IN (transitioning from 0 $\rightarrow$ 1) and INB (1 $\rightarrow$ 0), which serve as complementary WLs. This triggers the IBMD bitcells in the selected row to perform in-bitcell multiplication and digitization, inherently executing an AND logic operation between the row input (IN = 1) and the weight (W) stored in the MTJs. The resulting output (OUT) equals the stored W (Fig. 3b). Simultaneously, for unselected rows, the IN and INB signals remain at 0 and 1, respectively, ensuring that all the bitcell outputs of these rows are 0. Subsequently, the outputs of all bitcells within the same bank are aggregated by the adder tree (Fig. 3c). The adder tree sums these outputs and produces the final result, representing the stored data corresponding to the specified address.

In the compute mode, 256-input data are applied simultaneously to the nvDCIM banks in a most-significant-bit (MSB)-first, bit-serial manner (Methods and Extended Data Fig. 2a). Each nvDCIM bank performs 256 multiplications of 1-bit input and 4-bit weight in parallel during each cycle. For 4-bit precision inputs, this process costs four cycles. The results of these multiplications are then passed to a digital full-precision adder tree, which computes the partial sum (ADDOUT) of the 256 multiplications. A precision-reconfigurable accumulator manages these partial sums across cycles, supporting flexible input and weight precisions. This accumulator consists of multiple shift adders for reconfigurable input precision and a global adder tree for reconfigurable weight precision. Within the shift adder, the 12-bit ADDOUT is first extended to 28 bits to properly handle signed or unsigned weights, with signed weights stored in the two's-complement format. If the input data are signed and the current bit is in the MSB position, the bit-extended data undergo a negation by inverting all bits and adding 1 via the adder, as the signed input data are also represented in the two's-complement format. A 28-bit accumulator register (ACCOUT[27:0]) stores intermediate sums across multiple clock cycles, dynamically adjusting the input precision by shifting and accumulating over several cycles, which are equal to the input precision. In particular, when the current input data bit is in the MSB position, the accumulator register's output to the adder is set to zero by the AND gates, thereby restarting the accumulation process. The global adder tree then aggregates the outputs from the shift adders and aligns them based on the selected weight precision, which is controlled by the 2-bit weight-precision selection signals (WPS[1:0]). These signals determine the weight precision (4, 8, 12 or 16 bits). Each shift adder applies appropriate left shifts (for example, by 4, 8 or 12 bits) to align with the chosen

weight precision. Finally, the hierarchical adder tree sums the results, and a multiplexer ensures the output corresponds to the selected precision, achieving a flexible and efficient accumulation process.

The lossless computation and flexible precision of the nvDCIM macro enable software-equivalent inference accuracy across diverse AI workloads, including flow field prediction with PINNs, which demand high computational accuracy and bit precision, and image classification with convolutional neural networks (CNNs), such as ResNet-20 (ref. 49) for Canadian Institute For Advanced Research (CIFAR)-10 (ref. 50; Methods).

Extended Data Fig. 2b,c illustrates the architecture of the nvDCIM test chip and the chip-testing flow chart. The experimental and measurement platforms used to evaluate the nvDCIM chips are shown in Extended Data Fig. 3. The area breakdown of the main macro components is provided in Extended Data Fig. 4a, whereas the measured power consumption breakdown during ResNet-20 inference on the CIFAR-10 dataset is illustrated in Extended Data Fig. 4b. Extended Data Fig. 5 shows the measured shmoo plot, energy efficiency and test results obtained from 24 nvDCIM chips. A summary of the chip specifications, including detailed information and performance metrics, is provided in Extended Data Table 1. Variation analysis of the nvDCIM macro is provided in Supplementary Fig. 1 and Supplementary Note 2.

The nvDCIM macro achieves a peak MVM throughput of 4.42 TOPS at a V_{DD} of 1.20 V, operating at a frequency of 540 MHz, with an MVM area efficiency of 2.15 TOPS mm^{-2} when fully utilized for parallel MVM operations at 4-bit precision. Benefitting from the fully parallel computation, the measured MVM throughput per storage of 69.1 GOPS kb^{-1} (GOPS, giga-operations per second) is more than 3.04 times higher than the performance of previous state-of-the-art nvCIM works based on RRAM^{12,13,15}, MRAM^{18,41} and PCM²² (Extended Data Table 2). The macro achieves a peak energy efficiency of $7.02\text{--}112.3\text{ TOPS W}^{-1}$ at $V_{DD} = 0.65\text{ V}$ operating at 80 MHz, when the weight sparsity is 50% and the input toggle rate is 6.25%. When processing the ResNet-20 model at 8-bit precision, a peak MVM energy efficiency of 9.1 TOPS W^{-1} (boosted to 18.9 TOPS W^{-1} with our toggle-rate-aware training scheme) at 0.85 V is demonstrated, with a software-equivalent accuracy of 92.51%. Additional discussions, including the impact of the STT-MRAM-based nvDCIM macro on secure AI edge applications and its use in binarized NNs, are provided in Supplementary Figs. 2–4 and Supplementary Notes 3 and 4.

For flow field prediction (Methods), a PINN⁵¹ model at 16-bit precision (Extended Data Fig. 6 shows the PINN model quantization and performance evaluation), consisting entirely of fully connected layers with maximum dimensions of 32×32 , is deployed on part of the nvDCIM macro for MVM operations. This deployment achieves a peak MVM energy efficiency of 1.5 TOPS W^{-1} at 0.85 V. It delivers flow field predictions with relative L2 norm (R_{L2}) values of 0.027 for u (streamwise velocity), 0.054 for v (spanwise velocity) and 0.084 for p (pressure), matching the accuracy of the software baseline.

Toggle-rate-aware training scheme

The energy efficiency of NNs operating on CIM systems is critical for enabling their large-scale deployment. Our experimental measurements reveal that over 90% of the power consumption in our nvDCIM macro stems from three primary components: IBMD bitcells, adder trees and input drivers (Extended Data Fig. 4b). To further analyse and quantify the energy consumption of these components, a comprehensive energy model was developed for the nvDCIM macro. This model integrates circuit-level energy estimation with hardware mapping strategies (Fig. 4a) to evaluate the energy consumption of convolution and fully connected layers in NN workloads (Methods and Extended Data Fig. 7). For convolution operations, the input feature maps are restructured using image-to-column (im2col) transformation, facilitating efficient MVM operations with kernel weights. These kernel weights are distributed across several memory banks, enabling parallel processing within the nvDCIM macros.

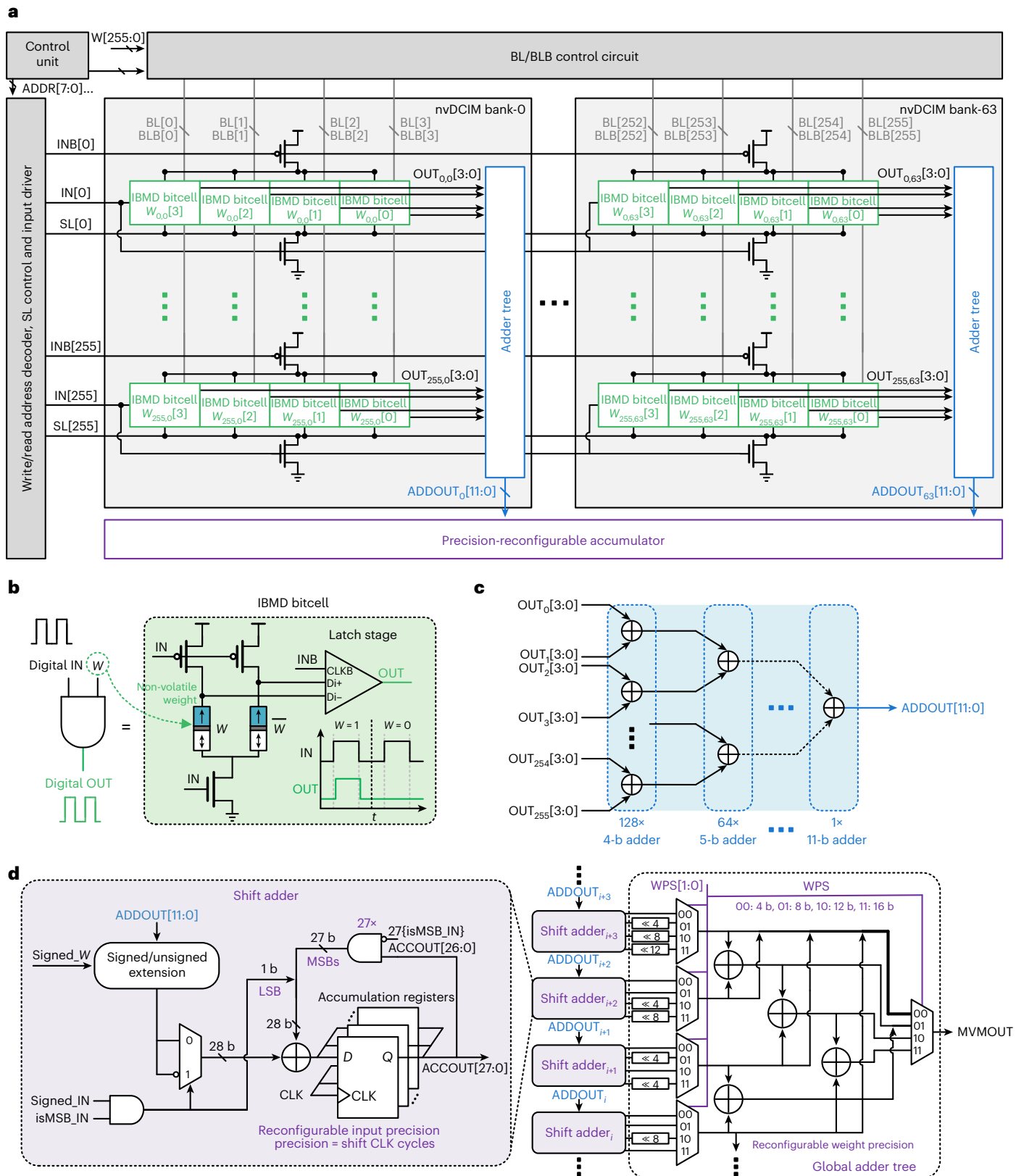


Fig. 3 | System architecture of the nvDCIM macro. a, Overall macro architecture includes the nvDCIM bank, control circuit, write circuit, address decoder and a precision-reconfigurable accumulator. **b**, Simplified schematic of the IBMD bitcell that performs an AND operation between digital inputs and non-volatile weights stored in the STT-MRAM, producing a digital output. **c**, Adder tree sums 256×4 -bit outputs from the IBMD-bitcell array into a 12-bit result (ADDOUT). **d**, Detailed design of the precision-reconfigurable accumulator and global adder tree, supporting multibit (up to 16 bits) signed and unsigned inputs and

weights, with configurable input and weight precisions for flexible and efficient computation. LSB, least-significant bit; *D*, input of the accumulation registers; *Q*, output of the accumulation registers; WPS, weight-precision selection signal; MVMOUT, final MVM output. Signed_*W* and Signed_*IN* indicate whether the weight and input are signed (1) or unsigned (0); isMSB_*IN* indicates whether the current input bit is the MSB (1) or lower bit (0) of the multibit input data during that clock cycle.

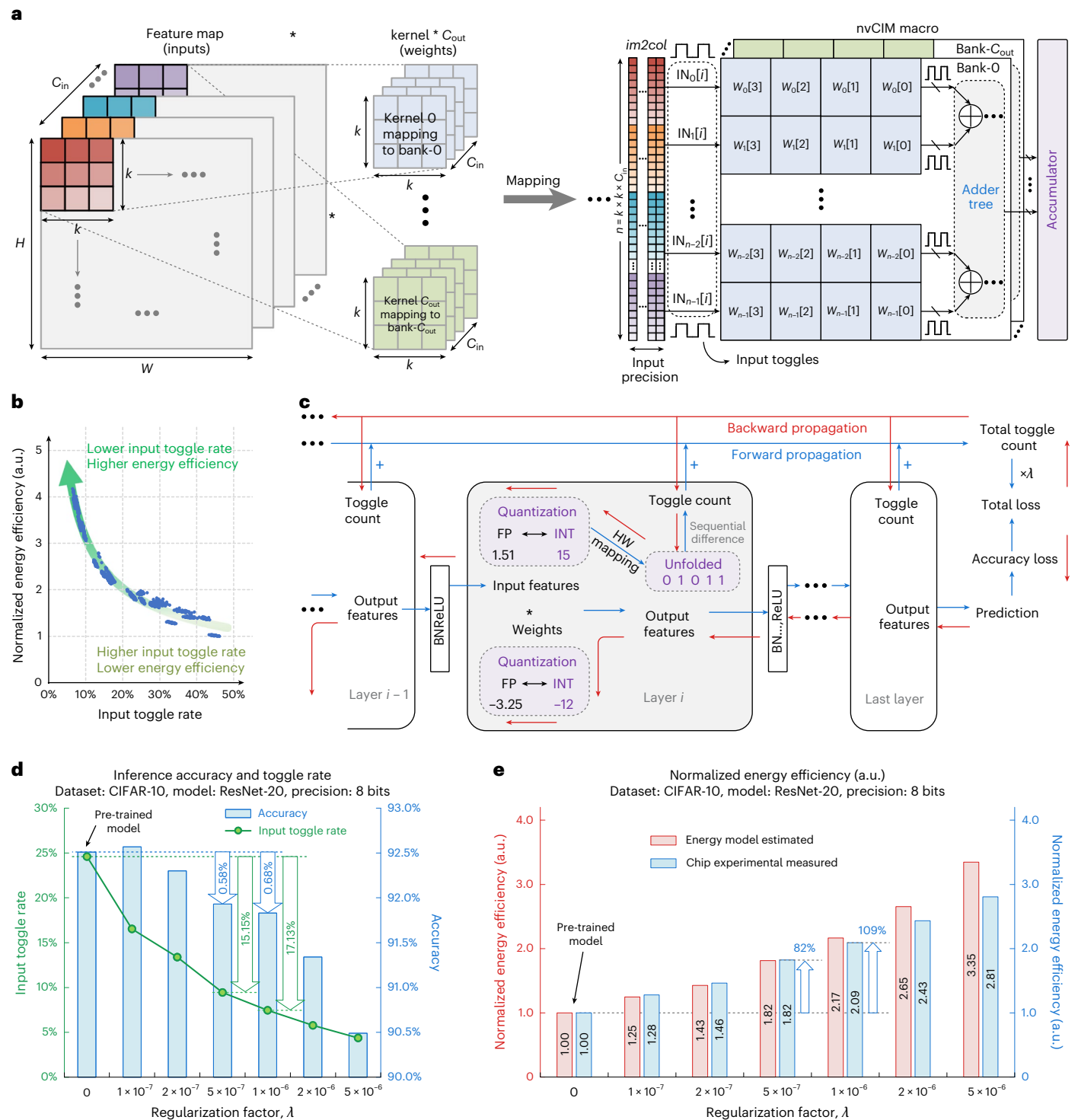


Fig. 4 | Toggle-rate-aware training scheme. a, Illustration of the convolution operation in which the input feature map is processed through convolutional kernels mapped to multiple banks in the nvDCIM macro. **b**, Per-layer input toggle rate versus energy efficiency for a quantized INT8 ResNet-20 model, evaluated on 50 random images from the validation dataset. The green arrow illustrates the overall trend that layers with lower input toggle rates tend to exhibit higher energy efficiency. **c**, Toggle-rate-aware training scheme. λ is a regularization

factor that controls the contribution of the total toggle count relative to the accuracy loss in the total loss function. FP, floating point; INT, integer; HW, hardware; ReLU, rectified linear unit. **d**, Compared with the pretrained model, the trade-off between the trained toggle rate, accuracy and λ is shown. **e**, Energy efficiency comparison between pretrained and post-toggle-rate-aware training models is evaluated using energy estimation models and measurements on the nvDCIM macro.

Using this energy model, we analysed the factors influencing the dynamic power dissipation of the nvDCIM macro. A key finding is the direct impact of the input toggle rate—the frequency of bit transitions in the input signals—on the macro’s energy consumption. The dynamic power of CMOS digital systems can be calculated by

$$P_{\text{dynamic}} = \alpha C V_{\text{DD}}^2 f, \quad (2)$$

where α , C , V_{DD} and f represent the switching activity, total capacitance, supply voltage and operating frequency, respectively. Hence, reducing the toggle rate of input signals directly decreases the dynamic

power dissipation of a digital system⁵². Our experimental measurements further validate this relationship, demonstrating that a lower input toggle rate results in higher energy efficiency for the nvDCIM macro (Extended Data Fig. 5b). Similar trends are also observed in other DCIM macros^{33–38}. Moreover, the energy model reveals that when mapping MVM operations in NN workloads, such as those in convolutional layers, onto the nvDCIM macro, the energy consumption is susceptible to the input toggle rate (Fig. 4b).

Hence, we implemented a hardware–software co-optimization method to bridge the gap between algorithm design and energy-efficient hardware implementation. Specifically, we introduced a toggle-rate-aware training scheme that explicitly minimizes the input toggle rate during the training process, thereby enabling more energy-efficient executions of NNs on the nvDCIM macro. This approach built on NNs already optimized for high accuracy and energy efficiency through quantization-aware training (QAT; Methods), further enhancing their energy efficiency.

As depicted in Fig. 4c, during forward propagation, floating-point 32-bit (FP32) feature maps are quantized into integer (INT) representations and subsequently unfolded into matrix formats compatible with the nvDCIM macro. The toggle count in the hardware input is calculated by summing the bit-wise sequential differences (for example, through XOR operations) between consecutive vectors. At the end of forward propagation, the total toggle count across the entire network is obtained. To optimize the system's energy efficiency, the total toggle count is minimized as a regularization term in the loss function:

$$L_{\text{total}} = L_{\text{accuracy}} + \lambda \times L_{\text{toggle}}, \quad (3)$$

where L_{accuracy} represents the original task-specific loss function (for example, cross-entropy for classification), L_{toggle} represents the total toggle count of input signals across the entire network and λ is a regularization factor that balances the trade-off between task accuracy and energy efficiency. The computation of the total toggle count is explicitly designed to be differentiable to the network parameters, ensuring compatibility with gradient-based optimization methods (Methods). This design enables backward propagation to minimize toggle activity effectively and preserve task performance.

During backward propagation, the gradient of L_{toggle} is calculated to penalize a high input toggle rate, thereby guiding the network to generate feature maps that naturally result in a lower toggle rate when mapped to the nvDCIM macro. As a result, the toggle-rate-aware training method yields networks that are not only accurate but also optimized for energy-efficient deployment on DCIM-based AI processors.

To assess the effectiveness and performance of this training scheme, experiments are conducted on representative NN workloads, specifically CNNs, such as the ResNet-20 model for CIFAR-10 image classification at 8-bit precision (Extended Data Fig. 8 shows more results). As shown in Fig. 4d, toggle-rate-aware training leads to an increasing reduction in the input toggle rate with increasing values of λ . Starting from an initial toggle rate of 24.59% in the pretrained model, the toggle rate is reduced to 9.44% and 7.46% for λ values of 5×10^{-7} and 1×10^{-6} , respectively, with corresponding negligible accuracy degradations of 0.58% and 0.68%. Furthermore (Fig. 4e), this reduction in the toggle rate directly translates to improvements in energy efficiency for the nvDCIM macro. The on-chip energy efficiency improves by 82% and 109% for λ values of 5×10^{-7} and 1×10^{-6} , respectively, closely matching estimations from our energy model.

Conclusions

We have reported a 64-kb CMOS-integrated spintronic nvDCIM macro that can perform lossless and fully parallel MVM operations with short latency, high energy efficiency, high throughput and flexible precision for AI applications. We designed an IBMD scheme for resistive non-volatile memory, which was integrated into a fully digital

architecture with digital adder trees and reconfigurable accumulators. Our nvDCIM macro performs MVM at an energy efficiency of 7.02–112.3 TOPS W^{-1} across precision configurations ranging from 4 to 16 bits. The measured MVM throughput per storage of 69.1 GOPS kb^{-1} is more than 17 times higher than a 64-core mixed-signal nvCIM chip²² based on phase-change memory. We measured a MVM throughput per area of 2,150 GOPS mm^{-2} , which is over 44 times higher than a 6.6-Mb nvCIM chip⁴¹ that uses 22-nm STT-MRAM technology. With its lossless MVM operation, our macro achieves CIFAR-10 and CIFAR-100 accuracies of 92.51% and 65.60%, respectively, and PINN flow field prediction R_{L_2} of 0.027 for u , 0.054 for v and 0.084 for p (where u and v are velocity components or p is the pressure), identical to the software baseline. The toggle-rate-aware training method improves the energy efficiency of the macro when deploying NN models, and offered an improvement of up to 109% with only 0.68% accuracy degradation on ResNet-20.

Methods

STT-MRAM device information

The MRAM used in our work is based on a current-driven MTJ architecture, consisting of two ferromagnetic layers separated by a thin insulating barrier. It is integrated within CMOS technology between layers Metal-6 and Metal-7 (Fig. 1b), as suggested by the foundry. However, the MTJs can actually be inserted between any two metal layers, depending on the foundry's process or cell size requirements. Extended Data Fig. 1a shows the vertical structure of the foundry-provided MTJ device. One of the ferromagnetic layers serves as the RL with a fixed magnetization, whereas the other acts as the free layer, whose magnetization can switch between P and AP orientations relative to the RL. These states correspond to distinct resistance levels— R_L in the P configuration and R_H in the AP configuration. Additionally, the device incorporates a hard layer, which exhibits strong perpendicular magnetic anisotropy and forms a synthetic antiferromagnetic⁵³ structure with the RL. Extended Data Fig. 1b shows the measured current–voltage characteristics of the foundry-provided MTJ device. The cross-sectional scanning electron microscopy image reveals the MTJ's critical diameter to be 78 nm. The resistance–voltage curve (Extended Data Fig. 1c) highlights the magnetic resistive switching of the MTJ, demonstrating a clear resistance transition between the high-resistance (AP orientations) and low-resistance (P orientations) states. The tunnel magnetoresistance (TMR) of the MTJ is approximately 170% at 0.1 V, calculated as $\text{TMR} = (R_{\text{AP}} - R_{\text{P}})/R_{\text{P}} \times 100\%$, where R_{AP} and R_{P} denote the MTJ resistances in the AP and P states, respectively.

In addition, variations in the STT-MRAM properties were experimentally characterized using the fabricated wafer. Key parameters, including state-switching voltages ($V_{\text{P} \rightarrow \text{AP}}$, $V_{\text{AP} \rightarrow \text{P}}$), resistance (R_{AP} , R_{P}), TMR and their variations were measured. Extended Data Fig. 1d shows the distribution of switching voltages, with a mean $V_{\text{AP} \rightarrow \text{P}}$ of 0.461 V ($\sigma = 0.029$ V; coefficient of variation, CV = 6.3%) and a mean $V_{\text{P} \rightarrow \text{AP}}$ of -0.299 V ($\sigma = 0.020$ V, CV = 6.7%). Extended Data Fig. 1e illustrates the resistance distributions, where the mean R_{AP} is 9,199.2 Ω ($\sigma = 480.7$ Ω , CV = 5.2%) and mean R_{P} is 3,363.7 Ω ($\sigma = 171.8$ Ω , CV = 5.1%). The TMR distribution (Extended Data Fig. 1f) has a mean of 173.5% with a standard deviation of 3.9% (CV = 2.2%). These results demonstrate small variations in switching voltages and TMR, along with a high resistance margin: $\mu(R_{\text{AP}}) - \mu(R_{\text{P}}) \approx 34\sigma(R_{\text{P}})$, highlighting the device's robustness and suitability for integration into advanced systems. Analysis of the challenges associated with STT-MRAM is provided in Supplementary Note 5.

MVM timing diagram, test chip architecture and test flow chart

The MVM timing diagram of the nvDCIM macro (Extended Data Fig. 2a) is best understood in conjunction with the circuit diagram shown in Fig. 3a,d. It shows the bit-serial processing of inputs and weights, beginning with the MSB and progressing to lower-order bits. Two examples are highlighted: the first MVM uses a 4-bit unsigned weight

and a 4-bit signed input, whereas the second uses an 8-bit signed weight and a 4-bit unsigned input. During the first MVM, the signed W signal is held at logic low (0) to represent the unsigned weight, whereas signed IN remains at logic high (1) for the signed input. The $isMSB_IN$ signal is set to logic high (1) during the first clock cycle and transitions to logic low (0) in subsequent cycles. By leveraging the pipelined architecture, the result of the first MVM is available at the start of the second MVM. The input data ($IN[255:0]$) and control signals ($isMSB_IN$, signed IN , signed W) are clock-synchronized to ensure precise and efficient computation.

Extended Data Fig. 2b shows the architecture of the nvDCIM test chip, with the previously introduced nvDCIM macro at its core. The chip includes a full-speed MVM evaluation testing harness. Key components of the testing harness include integrated on-chip buffers for input and output data storage, a clock generation unit and a serial–parallel interface for transferring data flow to and from the chip. Input–output interfaces and chip bonding pads (PADs) enable precise control of the chip and power supply by external testing instruments.

Extended Data Fig. 2c outlines the test flow of the nvDCIM test chip. The process begins by writing weight data into the MTJs and then reading and verifying the stored data to ensure write accuracy. This step is repeated 256 times to write and verify all rows of the bitcell array. If the read data do not match the written data (which is rare given the low write error rate of MTJs⁵⁴), the system identifies the specific rows with errors and rewrites them to ensure data accuracy. After this, the input data are loaded and written into the input buffer. The chip is subsequently configured, including settings for computation precision and data sign. Next, the clock frequency (f_{CLK}) and macro supply voltage (V_{DD}) are initialized, starting with an f_{CLK} based on simulation estimates. During computation, the power consumed by the macro is measured and recorded using source measurement units. Once the computation is completed, the macro computation results are read from the buffer and compared with the target values. If discrepancies are detected, f_{CLK} is adjusted and the same V_{DD} is maintained, and this computation and verification cycle is repeated until the results match the target values. At this point, the final results and the corresponding f_{CLK} and V_{DD} settings are recorded.

To comprehensively characterize the nvDCIM macro, the chip configuration, V_{DD} , computing frequency (f_{CLK}) and data sparsity or toggle rate of the input and weight data are systematically varied. The testing flow is repeated under these controlled conditions to map the macro's full performance spectrum.

Power and area breakdown

Extended Data Fig. 4a provides an area breakdown of our 64-kb nvDCIM macro, revealing that the IBMD bitcells and adder trees account for only 18.2% of the total macro area. Similarly, Extended Data Fig. 4b provides a power breakdown, showing that over 90% of the total power is consumed by the input driver, bitcells and adder tree, whereas the precision-reconfigurable accumulator and control unit account for just 8.23% of the power consumption.

Experimental platform and measurement result

Our 64-kb nvDCIM macro was implemented on a test chip fabricated using a 40-nm CMOS logic process with foundry-provided STT-MRAM devices. The STT-MRAM cell array and all the peripheral circuits are fully integrated into a single chip. The chip was wire-bonded to a printed circuit board designed to interface with a larger test motherboard via a socket (Yamaichi NP89-44111). This modular design facilitates easy insertion and removal of the chipboard, simplifying chip replacement during testing. A Keysight B2902B source measure unit was used to supply the required voltages and measure currents. A National Instruments PXIe-6570 device generated the digital patterns for chip control and testing, in addition to sampling the serial–parallel interface output data from the chip (Extended Data Fig. 3a).

Additionally, PXIe-6570 was used to measure the supply voltage versus computing frequency and generate the corresponding shmoo plot (Extended Data Fig. 5a). The resolution of V_{DD} and the measured computing frequency f_{CLK} were configured at 0.05 V and 40 MHz, respectively.

Extended Data Fig. 5b illustrates the measured energy efficiency results from the nvDCIM macro. Energy efficiency measurements for 4-bit IN |4-bit W |16-bit OUT configurations demonstrate a range of 13.9 to 112.3 TOPS W^{-1} , achieved with 50% weight sparsity and input toggle rate between 50% and 6.25%.

In addition, the throughput and energy efficiency of the nvDCIM macros across 24 chips, sampled from 12 different shots on the same 12-inch wafer (two chips per shot), were measured. The sampling pattern (Extended Data Fig. 5c) followed a Z-shaped pattern to capture both centre and edge shots to account for process variations. A photograph of the fabricated wafer is shown in Extended Data Fig. 5d.

The nvDCIM macro's throughput and energy efficiency were measured during 4-bit MVMs under two supply voltages ($V_{DD} = 0.65$ V and 1.20 V) when the weight sparsity is 50% and the input toggle rate is 12.5%. The measured distributions are presented in Extended Data Fig. 5e–h. At $V_{DD} = 1.20$ V, the throughput increases, with a mean of 4.44 TOPS and a standard deviation of 0.10 TOPS (CV = 2.3%; Extended Data Fig. 5e). At $V_{DD} = 0.65$ V, the throughput exhibits a distribution with a mean of 0.64 TOPS and a standard deviation of 0.03 TOPS (CV = 4.7%; Extended Data Fig. 5f). At $V_{DD} = 1.20$ V, the energy efficiency decreases, with a mean of 40.1 TOPS W^{-1} and a standard deviation of 1.97 TOPS W^{-1} (CV = 4.9%; Extended Data Fig. 5g). At $V_{DD} = 0.65$ V, the energy efficiency shows a mean of 86.0 TOPS W^{-1} and a standard deviation of 8.82 TOPS W^{-1} (CV = 10.3%; Extended Data Fig. 5h). These results demonstrate stable throughput and energy efficiency across the sampled chips, with low variations under both low and high supply voltages.

QAT using CNN model

We trained multiple CNN models on various datasets to evaluate performance. Specifically, LeNet-5 (ref. 55) was used for the Modified National Institute of Standards and Technology database (MNIST)⁵⁶ dataset and ResNet-20 (ref. 49) was used for the CIFAR-10 and CIFAR-100 datasets⁵⁰.

MNIST comprises 70,000 greyscale images (28×28 pixel²) of handwritten digits (0–9), split into 60,000 training and 10,000 test samples. We use LeNet-5, a CNN architecture designed for digit recognition, for this task. CIFAR-10 and CIFAR-100 each contain 60,000 RGB images (32×32 pixel²), with 50,000 training and 10,000 test samples. CIFAR-10 has 10 classes, whereas CIFAR-100 includes 100 classes. ResNet-20, a compact residual network, is chosen for its efficiency in handling classification tasks.

Training was conducted on an NVIDIA RTX 4090D GPU using PyTorch (v. 2.3.0) and Python (v. 3.12.3). All models were initially trained for 240 epochs with a batch size of 128 and an initial learning rate of 1×10^{-2} , reduced by a factor of 0.1 every 60 epochs. The best test accuracy across epochs was reported as the final result.

QAT was applied to pretrained FP32 models to emulate low-precision arithmetic (for example, INT4 and INT8). QAT was conducted for five additional epochs with a learning rate of 1×10^{-5} using the same batch size. During QAT, weights and activations were quantized to the target precision during the forward pass. This quantization used a scale and zero point to map the FP32 values into the desired integer range. These integer values were then dequantized to FP32 for gradient computation in the backward pass. The overall process can be expressed as follows:

$$x_q = S \times \left(\text{round} \left(\frac{x}{S} + Z \right) - Z \right), \quad (4)$$

where x represents the original value, x_q represents the value after quantizing and dequantizing, S represents the scale and Z represents

the zero point. This process introduces quantization noise into the model during training.

To enable gradient flow through non-differentiable quantization operations (that is, round), the straight-through estimator was used, approximating the gradient of the quantized variable with respect to the original as 1. This approach injected quantization noise during training, enabling the network to learn robustness to reduced numerical precision.

QAT reduced the accuracy gap between the quantized and FP32 models. On CIFAR-10, the INT8 ResNet-20 model achieved 92.51% accuracy, closely matching the FP32 model's 92.96%. On CIFAR-100, the INT8 model achieved 65.60% versus 65.62% for FP32. For MNIST, the INT4 LeNet-5 model retained 98.57% accuracy, nearly identical to the FP32 baseline. These results highlight the effectiveness of QAT in maintaining model performance even when using reduced-precision computations.

PINN model's quantization and performance

To investigate the sensitivity of PINNs to reduced numerical precision, we evaluated the inference robustness of a PINN model⁵¹ applied to the Navier–Stokes equations, which describe the motion of fluids. The task involves reconstructing the flow field of a two-dimensional unsteady flow past a circular cylinder at a Reynolds number of 3,900.

The PINN architecture consisted entirely of fully connected layers. Quantization was applied to both weights and activations by computing scale factors and zero points based on their respective data ranges. Target bit widths included 4, 8, 12 and 16 bits, corresponding to the precision levels supported by the nvDCIM macro.

After quantization, the PINN model was trained on the original dataset using QAT. Inputs included spatial coordinates and time values, whereas outputs corresponded to the velocity components and pressure field. Training hyperparameters, including learning rate, batch size and number of epochs, were kept consistent with the baseline implementation to ensure fair comparison.

Model performance was evaluated across different quantization levels. Flow field predictions from quantized models (INT4 to INT16) were compared against the FP32 baseline and the computational fluid dynamics benchmark³² data. Qualitative comparisons (Extended Data Fig. 6a–f) visualize flow features. To quantitatively assess the deviation, R_{L2} , a widely used metric, is used. It is defined as

$$R_{L2} = \frac{\|\hat{U} - U\|}{U}, \quad (5)$$

where $\|\hat{U}\|$ represents the L2 norm of the prediction deviation of quantity (for example, u and v or p) at a given time. $\|U\|$ corresponds to the L2 norm of computational fluid dynamics at the same time. A lower R_{L2} value indicates higher prediction accuracy and better alignment with the original flow field. Extended Data Fig. 6g–i illustrates the temporal evolution of R_{L2} values for different model precisions and overall R_{L2} . These experiments provide insights into how computation precision affects the reliability and robustness of PINNs in solving complex physical problems, such as the Navier–Stokes equation. The results demonstrate that INT16 and INT12 PINN models achieve accuracy comparable to the FP32 model. By contrast, INT8 and INT4 models exhibit substantial degradation in both prediction accuracy and solution quality.

These findings underscore the critical importance of high computational precision in MVM for PINNs, as reduced precision substantially compromises both performance and stability.

Energy model construction

To quantify the energy consumption of the nvDCIM macro, a comprehensive energy model was developed (Extended Data Fig. 7). This model captured the dynamic power dissipation arising from both IBMD bitcells and peripheral circuits, including input drivers, adder trees and accumulators. The construction of the model involved several key steps.

First, the NN workload was systematically decomposed into its fundamental operations, including convolution and fully connected layers, whose MVM operations were then mapped onto the hardware architecture of the nvDCIM macro. The hardware mapping strategy ensured that feature maps (inputs) and kernels (weights) were appropriately assigned to the input drivers and memory banks.

Next, we analysed the energy consumption of the nvDCIM macro at the fundamental unit level, including the IBMD bitcells, full adders in the adder tree and input drivers. The input driver was implemented as a large inverter or buffer to handle the required drive strength. Energy dissipation for each unit was characterized using precomputed energy look-up tables derived from circuit-level simulations. These tables store the energy cost of specific operations, such as bit transitions within the IBMD bitcells (for example, from $IN[t-1]$ to $IN[t]$), the input drivers (for example, from $IN[t-1]$ to $IN[t]$) or full adders (for example, from $ABC_{in}[t-1]$ to $ABC_{in}[t]$).

By modelling the hardware in software—from fundamental units to the circuit level and then up to the macro level—and summing the energy consumption of all the fundamental units in the macro, the model accurately estimates the total dynamic energy consumption of MVM operations. Finally, integrating the low-level MVM energy estimation into a high-level NN model can simultaneously evaluate the accuracy and energy consumption of convolutional and fully connected layers.

Toggle-rate-aware training

To further optimize the models for hardware deployment, toggle-rate-aware training was introduced following QAT. This technique reduces bit-level switching activity during MVM operations, which is a dominant contributor to dynamic power consumption in DCIM hardware. The toggle rate, defined as the number of bit transitions (for example, $0 \rightarrow 1$ or $1 \rightarrow 0$), directly impacts the dynamic power consumption of digital circuits. Therefore, reducing the toggle rate is a fundamental objective in hardware-aware NN optimization.

During forward propagation, activations were quantized to low-bit formats (for example, INT4 or INT8). In convolutional layers, input feature maps were unfolded into sliding windows (for example, $im2col$) to enable MVM operations. Each unfolded input block was encoded into a binary sequence, and the toggle count was computed from the bit-wise differences (for example, XOR operations) between consecutive input vectors. Layer-wise toggle counts were obtained by summing transitions across all inputs.

As in QAT, the straight-through estimator was used during backward propagation to enable a gradient flow through non-differentiable quantization operations. As described in the main text, this training approach incorporates the total toggle count into the training objective, enabling the model to adapt its internal representations to minimize unnecessary bit-level transitions. By reducing these bit-level toggles, the approach substantially decreases the dynamic power consumption of hardware accelerators.

Following QAT, toggle-rate-aware training was performed for 240 additional epochs using the same learning rate schedule and batch size. To evaluate its effectiveness, experiments were conducted using INT4 LeNet-5 on MNIST and INT8 ResNet-20 on CIFAR-10 (Fig. 3d,e) and CIFAR-100 datasets. The results (Extended Data Fig. 8) demonstrate a trade-off between accuracy and input toggle rate, controlled by λ .

As λ increases, the input toggle rate decreases, improving energy efficiency. For instance, in the LeNet-5 experiment (Extended Data Fig. 8a), the toggle rate drops from 17.24% to 5.51% for $\lambda = 2 \times 10^{-5}$, with only a 0.55% accuracy loss. Similarly, for ResNet-20 (Extended Data Fig. 8c), the toggle rate decreases from 28.9% to 11.72% as $\lambda = 2 \times 10^{-6}$, with a 0.74% accuracy reduction. These reductions in toggle rate directly enhance the energy efficiency (Extended Data Fig. 8b,d). For LeNet-5, energy efficiency improves by 94% at $\lambda = 2 \times 10^{-5}$ with negligible accuracy degradation, whereas larger λ values yield more gains. For ResNet-20, energy efficiency improves by 70% at $\lambda = 2 \times 10^{-6}$, showing similar trends.

The results demonstrate that toggle-rate-aware training enables tunable trade-offs between accuracy and energy, offering a practical path for efficient deployment on nvDCIM-based AI processors.

Chip summary and comparison table

Extended Data Table 1 summarizes the chip, including detailed information and performance characteristics.

Extended Data Table 2 compares this work with previously reported nvCIM chips.

Data availability

The data that support the plots presented in this Article, as well as other findings derived from this study, are available from the corresponding authors upon reasonable request.

Code availability

Computer codes are available from the corresponding authors upon reasonable request.

References

- Sze, V., Chen, Y.-H., Yang, T.-J. & Emer, J. S. Efficient processing of deep neural networks: a tutorial and survey. *Proc. IEEE* **105**, 2295–2329 (2017).
- Xu, X. et al. Scaling for edge inference of deep neural networks. *Nat. Electron.* **1**, 216–222 (2018).
- Di Ventra, M. & Pershin, Y. V. The parallel approach. *Nat. Phys.* **9**, 200–202 (2013).
- Horowitz, M. 1.1 Computing's energy problem (and what we can do about it). In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 10–14 (IEEE, 2014).
- Yu, E., K, G. K., Saxena, U. & Roy, K. Ferroelectric capacitors and field-effect transistors as in-memory computing elements for machine learning workloads. *Sci. Rep.* **14**, 9426 (2024).
- Luo, Y.-C. et al. Experimental demonstration of non-volatile capacitive crossbar array for in-memory computing. In *Proc. IEEE International Electron Devices Meeting (IEDM)* 21.4.1–21.4.4 (IEEE, 2021).
- Slesazeck, S. et al. A 2TnC ferroelectric memory gain cell suitable for compute-in-memory and neuromorphic application. In *Proc. IEEE International Electron Devices Meeting (IEDM)* 38.6.1–38.6.4 (IEEE, 2019).
- Chen, W.-H. et al. CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors. *Nat. Electron.* **2**, 420–428 (2019).
- Cai, F. et al. A fully integrated reprogrammable memristor-CMOS system for efficient multiply-accumulate operations. *Nat. Electron.* **2**, 290–299 (2019).
- Yao, P. et al. Fully hardware-implemented memristor convolutional neural network. *Nature* **577**, 641–646 (2020).
- Lin, P. et al. Three-dimensional memristor circuits as complex neural networks. *Nat. Electron.* **3**, 225–232 (2020).
- Hung, J.-M. et al. A four-megabit compute-in-memory macro with eight-bit precision based on CMOS and resistive random-access memory for AI edge devices. *Nat. Electron.* **4**, 921–930 (2021).
- Xue, C.-X. et al. A CMOS-integrated compute-in-memory macro based on resistive random-access memory for AI edge devices. *Nat. Electron.* **4**, 81–90 (2021).
- Huo, Q. et al. A computing-in-memory macro based on three-dimensional resistive random-access memory. *Nat. Electron.* **5**, 469–477 (2022).
- Wan, W. et al. A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).
- Wen, T.-H. et al. Fusion of memristor and digital compute-in-memory processing for energy-efficient edge computing. *Science* **384**, 325–332 (2024).
- Cai, H. et al. A 28 nm 2 Mb STT-MRAM computing-in-memory macro with a refined bit-cell and 22.4–41.5 TOPS/W for AI inference. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 500–502 (IEEE, 2023).
- Jung, S. et al. A crossbar array of magnetoresistive memory devices for in-memory computing. *Nature* **601**, 211–216 (2022).
- Xie, W. et al. A 709.3 TOPS/W event-driven smart vision SoC with high-linearity and reconfigurable MRAM PIM. In *Proc. IEEE Symposium on VLSI Technology* 1–2 (IEEE, 2023).
- Deaville, P., Zhang, B. & Verma, N. A 22 nm 128-kb MRAM row/column-parallel in-memory computing macro with memory-resistance boosting and multi-column ADC readout. In *Proc. IEEE Symposium on VLSI Technology* 268–269 (IEEE, 2022).
- Joshi, V. et al. Accurate deep neural network inference using computational phase-change memory. *Nat. Commun.* **11**, 2473 (2020).
- Le Gallo, M. et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nat. Electron.* **6**, 680–693 (2023).
- Khaddam-Aljameh, R. et al. HERMES-core—a 1.59-TOPS/mm² PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs. *IEEE J. Solid-State Circuits* **57**, 1027–1038 (2022).
- Narayanan, P. et al. Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format. *IEEE Trans. Electron Devices* **68**, 6629–6636 (2021).
- Khwa, W.-S. et al. A 40-nm, 2M-cell, 8b-precision, hybrid SLC-MLC PCM computing-in-memory macro with 20.5–65.0 TOPS/W for tiny-AI edge devices. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 1–3 (IEEE, 2022).
- Sun, Z. et al. A full spectrum of computing-in-memory technologies. *Nat. Electron.* **6**, 823–835 (2023).
- Kim, H., Yoo, T., Kim, T. T.-H. & Kim, B. Colonnade: a reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks. *IEEE J. Solid-State Circuits* **56**, 2221–2233 (2021).
- Murmann, B. Mixed-signal computing for deep neural network inference. *IEEE Trans. Very Large Scale Integr. VLSI Syst.* **29**, 3–13 (2021).
- Murmann, B., Verhelst, M. & Manoli, Y. Analog-to-information conversion. In *NANO-CHIPS 2030: On-Chip AI for an Efficient Data-Driven World* 275–292 (Springer International Publishing, 2020).
- Murmann, B. A/D converter trends: power dissipation, scaling and digitally assisted architectures. In *Proc. IEEE Custom Integrated Circuits Conference (CICC)* 105–112 (IEEE, 2008).
- Jumper, J. et al. Highly accurate protein structure prediction with AlphaFold. *Nature* **596**, 583–589 (2021).
- Rudy, S. H., Brunton, S. L., Proctor, J. L. & Kutz, J. N. Data-driven discovery of partial differential equations. *Sci. Adv.* **3**, e1602614 (2017).
- Chih, Y.-D. et al. An 89 TOPS/W and 16.3 TOPS/mm² all-digital SRAM-based full-precision compute-in-memory macro in 22 nm for machine-learning edge applications. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 252–254 (IEEE, 2021).
- Mori, H. et al. A 4 nm 6163 TOPS/W/b 4790 TOPS/mm²/b SRAM-based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 132–134 (IEEE, 2023).

35. Fujiwara, H. et al. A 3 nm, 32.5 TOPS/W, 55.0 TOPS/mm² and 3.78 Mb/mm² fully-digital compute-in-memory macro supporting INT12×INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 572–574 (IEEE, 2024).
36. Shih, M.-E. et al. NVE: a 3 nm 23.2 TOPS/W 12b-digital-CIM-based neural engine for high-resolution visual-quality enhancement on smart devices. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 360–362 (IEEE, 2024).
37. Wang, J. et al. A 22 nm 29.3 TOPS/W end-to-end CIM-utilization-aware accelerator with reconfigurable 4D-CIM mapping and adaptive feature reuse for diverse CNNs and transformers. In *Proc. IEEE Custom Integrated Circuits Conference (CICC)* 1–3 (IEEE, 2025).
38. Lou, M. et al. Area-efficient and low-power 8T compute-SRAM bitcell design for digital compute-in-memory macros in 22 nm CMOS. *IEEE Trans. Circuits Syst. II Express Briefs* **72**, 1459–1463 (2025).
39. Lu, A. et al. High-speed emerging memories for AI hardware accelerators. *Nat. Rev. Electr. Eng.* **1**, 24–34 (2024).
40. Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
41. Chiu, Y.-C. et al. A CMOS-integrated spintronic compute-in-memory macro for secure AI edge devices. *Nat. Electron.* **6**, 534–543 (2023).
42. Yang, Z. et al. A novel computing-in-memory platform based on hybrid spintronic/CMOS memory. *IEEE Trans. Electron Devices* **69**, 1698–1705 (2022).
43. Sayadi, L., Amirany, A., Moaiyeri, M. H. & Timarchi, S. Balancing precision and efficiency: an approximate multiplier with built-in error compensation for error-resilient applications. *J. Supercomput.* **81**, 109 (2025).
44. Rezaei, M., Amirany, A., Moaiyeri, M. H. & Jafari, K. A reliable non-volatile in-memory computing associative memory based on spintronic neurons and synapses. *Eng. Rep.* **6**, e12902 (2024).
45. Angizi, S., He, Z., Chen, A. & Fan, D. Hybrid spin-CMOS polymorphic logic gate with application in in-memory computing. *IEEE Trans. Magn.* **56**, 3400215 (2020).
46. Tong, Z. et al. BSTCIM: a balanced symmetry ternary fully digital in-mram computing macro for energy efficiency neural network. *IEEE Trans. Circuits Syst. Regul. Pap.* **71**, 6114–6127 (2024).
47. Mazaheri, M. M., Amirany, A. & Moaiyeri, M. H. TPCSA-MRAM: ternary precharge sense amplifier-based MRAM. *IEEE Access* **12**, 132817–132824 (2024).
48. Rajaei, R. & Amirany, A. Nonvolatile low-cost approximate spintronic full adders for computing in memory architectures. *IEEE Trans. Magn.* **56**, 3400308 (2020).
49. He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 770–778 (IEEE, 2016).
50. Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images* (University of Toronto, 2009).
51. Xu, S. et al. A practical approach to flow field reconstruction with sparse or incomplete data through physics informed neural network. *Acta Mech. Sin.* **39**, 322302 (2023).
52. Chandrakasan, A. P. & Brodersen, R. W. Minimizing power consumption in digital CMOS circuits. *Proc. IEEE* **83**, 498–523 (1995).
53. Natarajathinam, A., Zhu, R., Visscher, P. B. & Gupta, S. Perpendicular magnetic tunnel junctions based on thin CoFeB free layer and Co-based multilayer synthetic antiferromagnet pinned layers. *J. Appl. Phys.* **111**, 07C918 (2012).
54. Song, J., Dixit, H., Behin-Aein, B., Kim, C. H. & Taylor, W. Impact of process variability on write error rate and read disturbance in STT-MRAM devices. *IEEE Trans. Magn.* **56**, 3400411 (2020).
55. Lecun, Y., Bottou, L., Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324 (1998).
56. Deng, L. The MNIST database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Process. Mag.* **29**, 141–142 (2012).
57. Yang, J. et al. TIMAQ: a time-domain computing-in-memory-based processor using predictable decomposed convolution for arbitrary quantized DNNs. *IEEE J. Solid-State Circuits* **56**, 3021–3038 (2021).
58. Jain, S., Lin, L. & Alioto, M. ±CIM SRAM for signed in-memory broad-purpose computing from DSP to neural processing. *IEEE J. Solid-State Circuits* **56**, 2981–2992 (2021).
59. Yoshioka, K. A 818–4,094 TOPS/W capacitor-reconfigured CIM macro for unified acceleration of CNNs and transformers. In *Proc. IEEE International Solid-State Circuits Conference (ISSCC)* 574–576 (IEEE, 2024).

Acknowledgements

This work was supported by the National Key R&D Program of China (grant number 2022YFB4400200 to T.M.), the National Natural Science Foundation of China (grant numbers 62274081 to L.L. and 12327806 to Z.C.), Zhujiang Young Talent Program (grant number 2023QN10X177 to L.L.) and Shenzhen Stable Support Plan Program for Higher Education Institutions Research Program (grant number 20231121110457002 to L.L.). We acknowledge the SUSTech SME-Pixelcore Neuromorphic In-sensor Computing Joint Laboratory and the SUSTech SME-CIMCube Joint Laboratory for experimental support in this work.

Author contributions

L.L. and T.M. conceived and supervised the project. H. Li, R.P. and L.L. designed the circuits for the nvDCIM macro and test chip. H. Li, W.D. and J.H. performed the training, quantization and inference of the NNs and implemented the toggle-rate-aware training algorithm. H. Li, Z.C., S.L., Z.K., X.Y., X.W., Z.Y., H. Lyu, H.Y. and X.Z. performed the experiments, including device characterization and building the chip-testing platform, and chip testing. H. Li, Z.C., J.L., F.Z., Y.L., Z.X., T.M. and L.L. analysed the data. H. Li, Z.C., T.M. and L.L. wrote the paper. All authors reviewed and approved the paper.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s41928-025-01479-y>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41928-025-01479-y>.

Correspondence and requests for materials should be addressed to Tai Min or Longyang Lin.

Peer review information *Nature Electronics* thanks Abdollah Amirany, Esteban Garzón and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

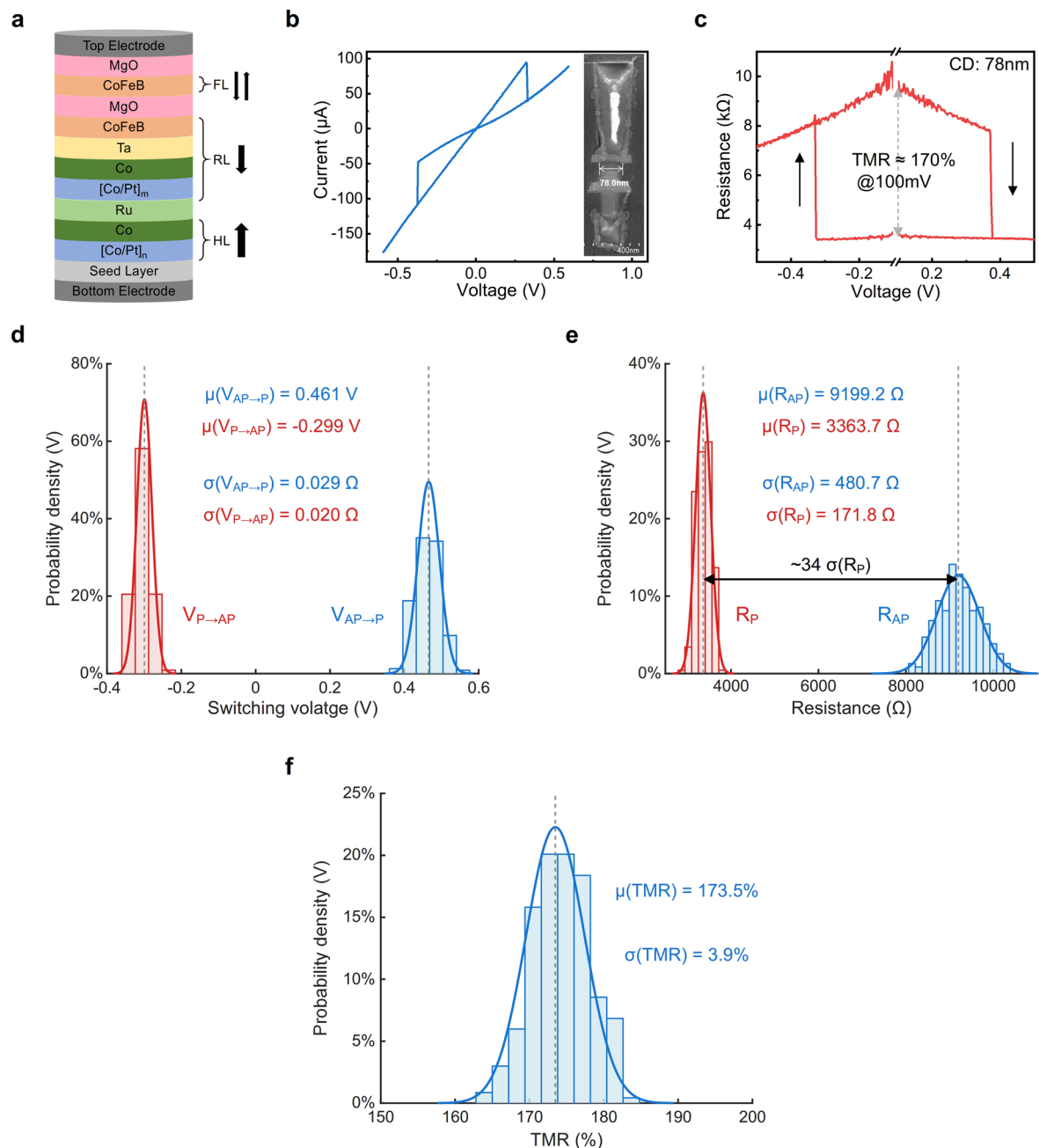
Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with

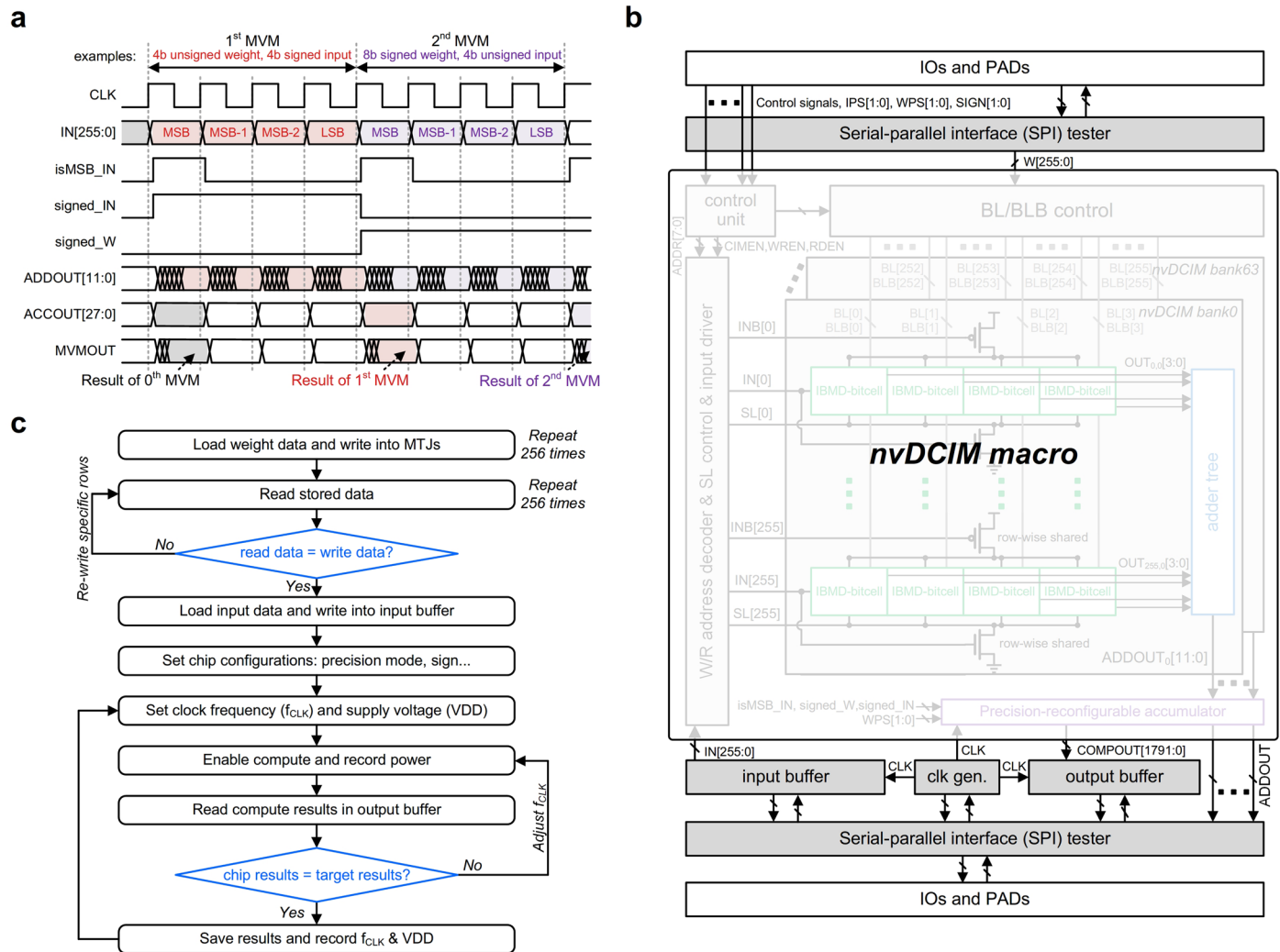
the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2025



Extended Data Fig. 1 | STT-MRAM device. (a) Vertical structure of MTJ. RL: reference layer, which has a fixed magnetization. FL: free layer, whose magnetization can switch between parallel (P) and antiparallel (AP) orientations relative to the reference layer. HL: hard layer, which possesses strong perpendicular magnetic anisotropy (PMA). (b) Measured I-V curve of the MTJ's magnetic resistive switching. The SEM image shows the MTJ's critical diameter of 78 nm. (c) Measured R-V curve of the MTJ's magnetic resistive switching, showing a clear resistance change between high-resistance (anti-parallel, AP) and low-resistance (parallel, P)

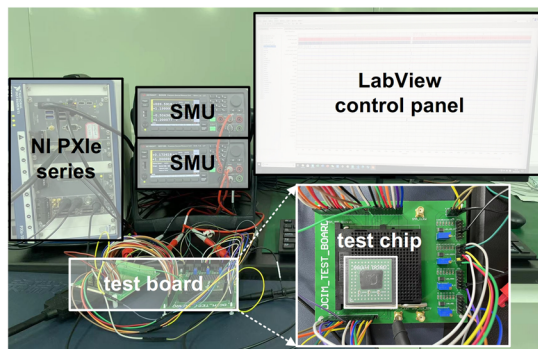
states. The TMR ratio is approximately 170% at 0.1 V. TMR: Tunnel Magneto-Resistance, $\text{TMR} = (R_{AP} - R_P) / R_P \times 100\%$. (d) Measured distribution of state switching voltages ($V_{AP \rightarrow P}$, $V_{P \rightarrow AP}$), with a mean $V_{AP \rightarrow P}$ of 0.461 V (standard deviation $\sigma = 0.029$ V, CV = 6.3%) and a mean $V_{P \rightarrow AP}$ of -0.299 V ($\sigma = 0.020$ V, CV = 6.7%). (e) Resistance distributions of the R_P and R_{AP} , where the mean R_{AP} is 9199.2 Ω ($\sigma = 480.7$ Ω, CV = 5.2%) and the mean R_P is 3363.7 Ω ($\sigma = 171.8$ Ω, CV = 5.1%). (f) Measured TMR distribution, with a mean of 173.5% ($\sigma = 3.9\%$, CV = 2.2%). CV: Coefficient of Variation, CV = standard deviation (σ) / mean (μ) * 100%.



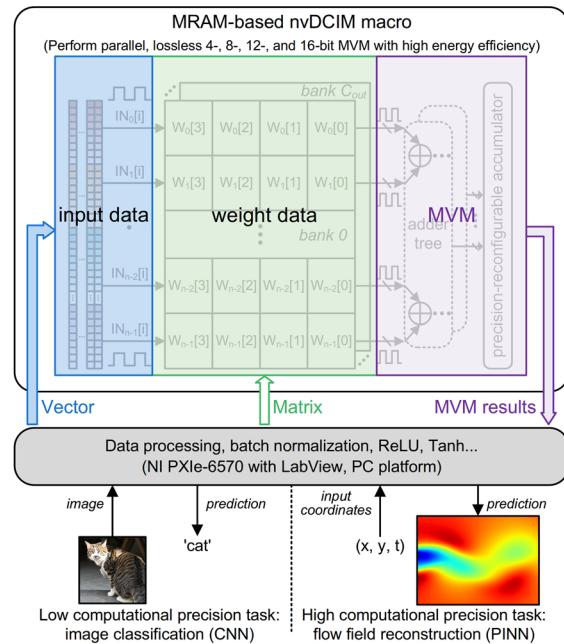
Extended Data Fig. 2 | MVM timing diagram, test chip architecture, and test flow chart. (a) The nvDCIM test chip architecture featuring the nvDCIM macro, integrated on-chip buffers, a clock generator, and SPI interfaces for data transfer.

(b) nvDCIM chip test flow chart. **(c)** MVM timing diagram illustrating bit-serial processing for two examples: 4-bit unsigned weight with 4-bit signed input and 8-bit signed weight with 4-bit unsigned input.

a

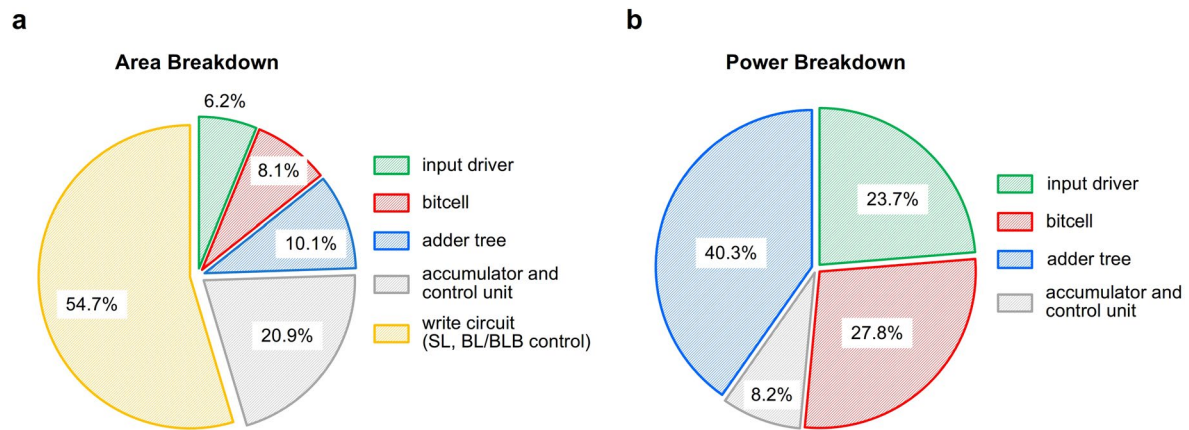


b

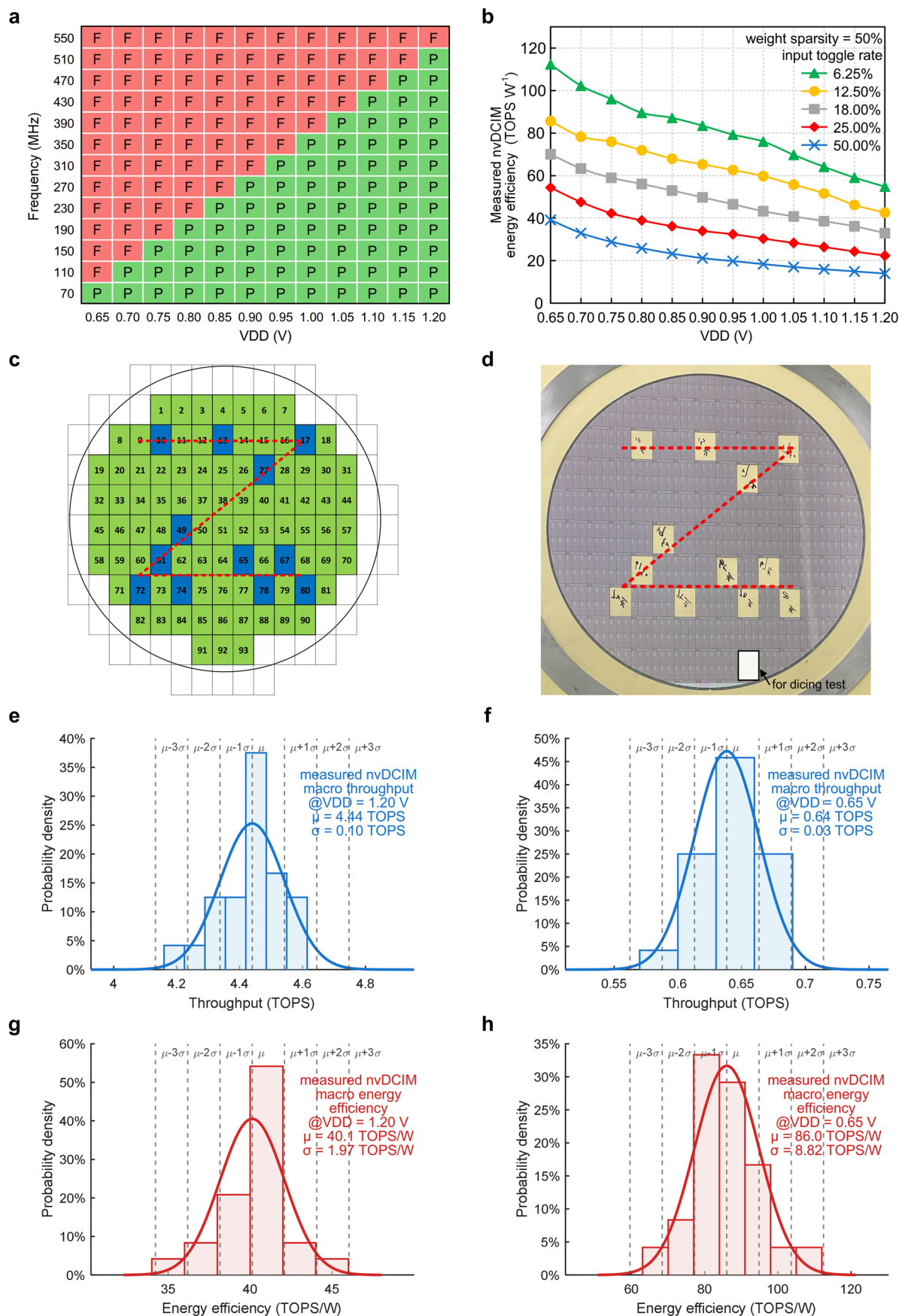


Extended Data Fig. 3 | Experimental and measurement platform for evaluating the nvDCIM chips. (a) The experimental platform consists of an nvDCIM test chip, a PCB test board, and a National Instruments (NI) PXIe system, including the PXIe-6570 and PXIe-8881 modules, which handle chip control, intermediate data processing, and result visualization. Additionally, two source/measurement units (SMUs) are included in the platform for power measurements. (b) A flowchart illustrating the inference process conducted on the experimental platform. During the execution, the 64-kb nvDCIM macro performs parallel and lossless MVM operations across 4-, 8-, 12-, and 16-bit precisions for convolutional and

fully connected layers. Input vectors and matrix data are supplied to the nvDCIM macro via the NI PXIe-6570 controlled through LabVIEW, which also retrieves the MVM results. Beyond this, the PXIe-6570 implements ReLU activations, pooling, Tanh activations, and batch normalization. The PXIe-8881 processes and displays the final inference results. The system supports a range of computational tasks, such as low computational precision tasks (for example, image classification with CNNs) and high computational precision tasks (for example, flow field reconstruction using PINNs).



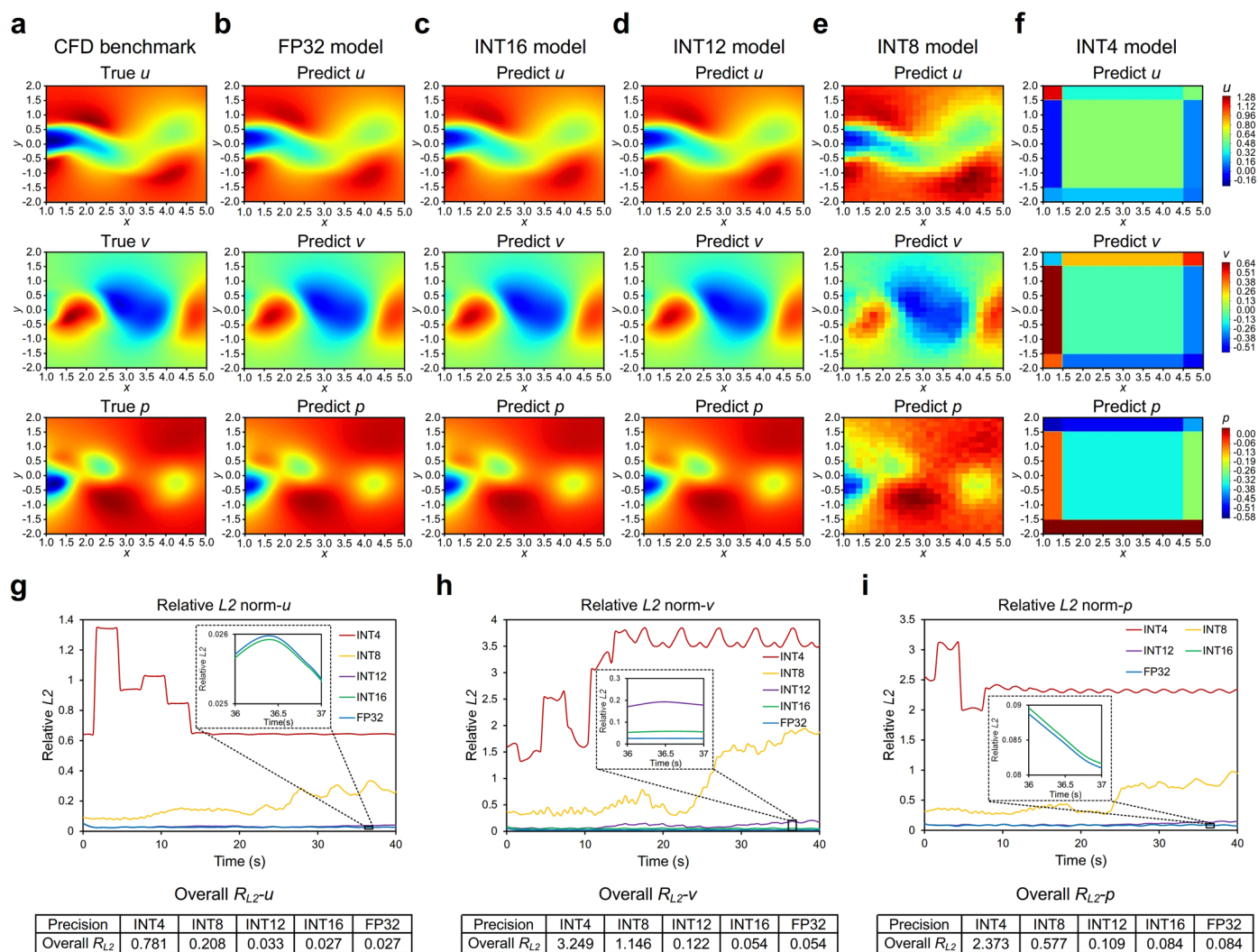
Extended Data Fig. 4 | Power and area breakdown. (a) Area breakdown of the main macro components. (b) Power consumption breakdown of the nvDCIM macro components measured during the inference of the ResNet-20 model on the CIFAR-10 dataset.



Extended Data Fig. 5 | See next page for caption.

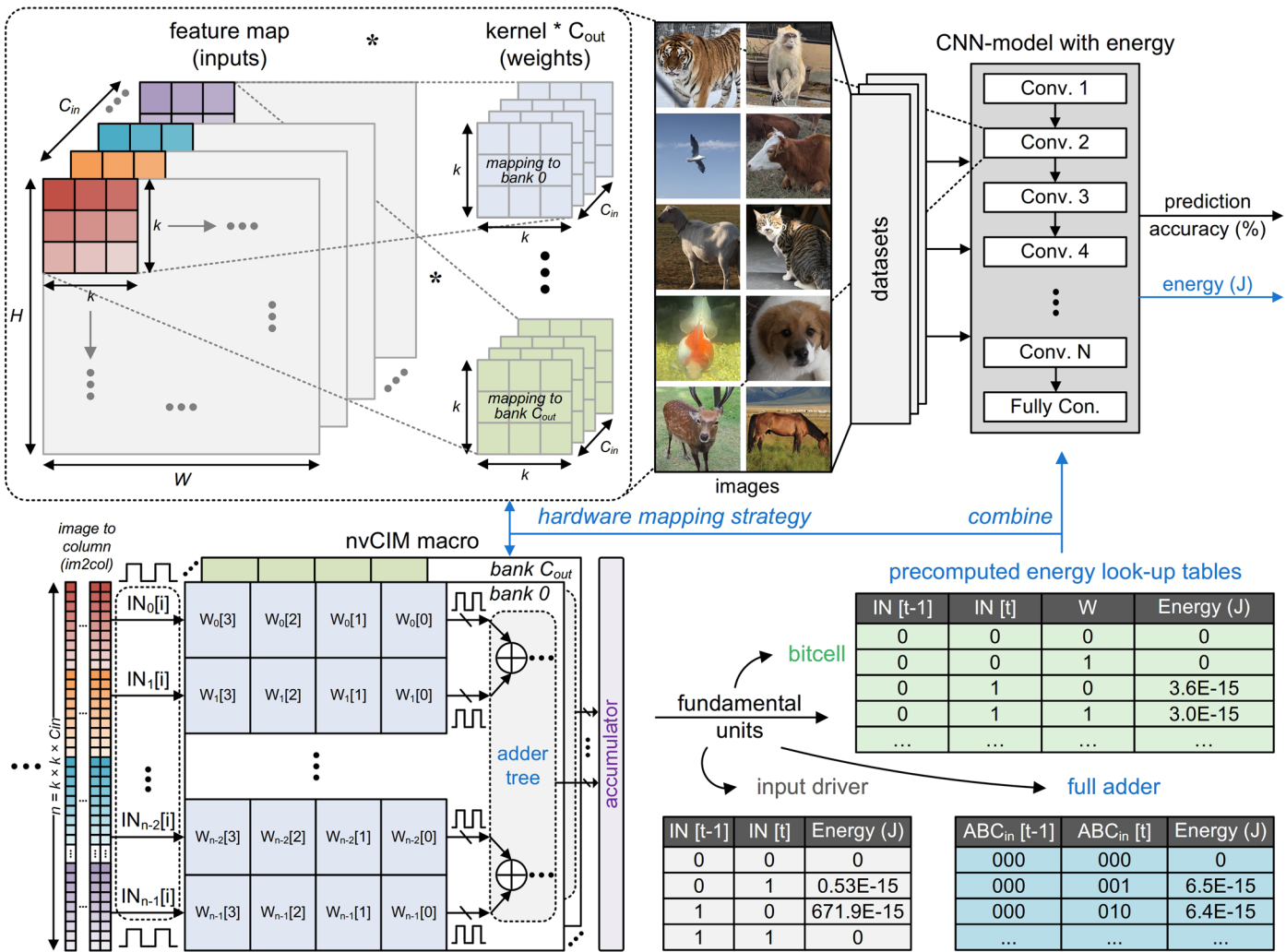
Extended Data Fig. 5 | Measured shmoo plot, energy efficiency, and test results across 24 nvDCIM chips. (a) Measured shmoo plot of the nvDCIM macro showing the relationship between supply voltage (VDD) and maximum clock frequency (f_{clk}) while operating in 4-bit-input, 4-bit-weight, and 16-bit-output mode. (b) Measured energy efficiency of the nvDCIM macro versus VDD in 4-bit-input, 4-bit-weight, and 16-bit-output mode, when the weight sparsity is 50% and the input toggle rate ranges from 50% to 6.25%. F, Fail; P, Pass. (c) Wafer map showing 12 selected shots (highlighted in blue), with the Z-pattern sampling. (d) Photograph of the fabricated 12-inch wafer, showing the positions of

the selected shots, corresponding to the Z-pattern used in (c). (e) Measured throughput (TOPS) distribution at $VDD = 1.20$ V, with a mean of 4.44 TOPS (standard deviation $\sigma = 0.10$ TOPS, CV = 2.3%). (f) Measured throughput (TOPS) distribution at $VDD = 0.65$ V, with a mean of 0.64 TOPS ($\sigma = 0.03$ TOPS, CV = 4.7%). (g) Measured energy efficiency (TOPS/W) distribution at $VDD = 1.20$ V, with a mean of 40.1 TOPS/W ($\sigma = 1.97$ TOPS/W, CV = 4.9%). (h) Measured energy efficiency (TOPS/W) distribution at $VDD = 0.65$ V, with a mean of 86.0 TOPS/W ($\sigma = 8.82$ TOPS/W, CV = 10.3%). CV: Coefficient of Variation, CV = standard deviation (σ) / mean (μ) * 100%.



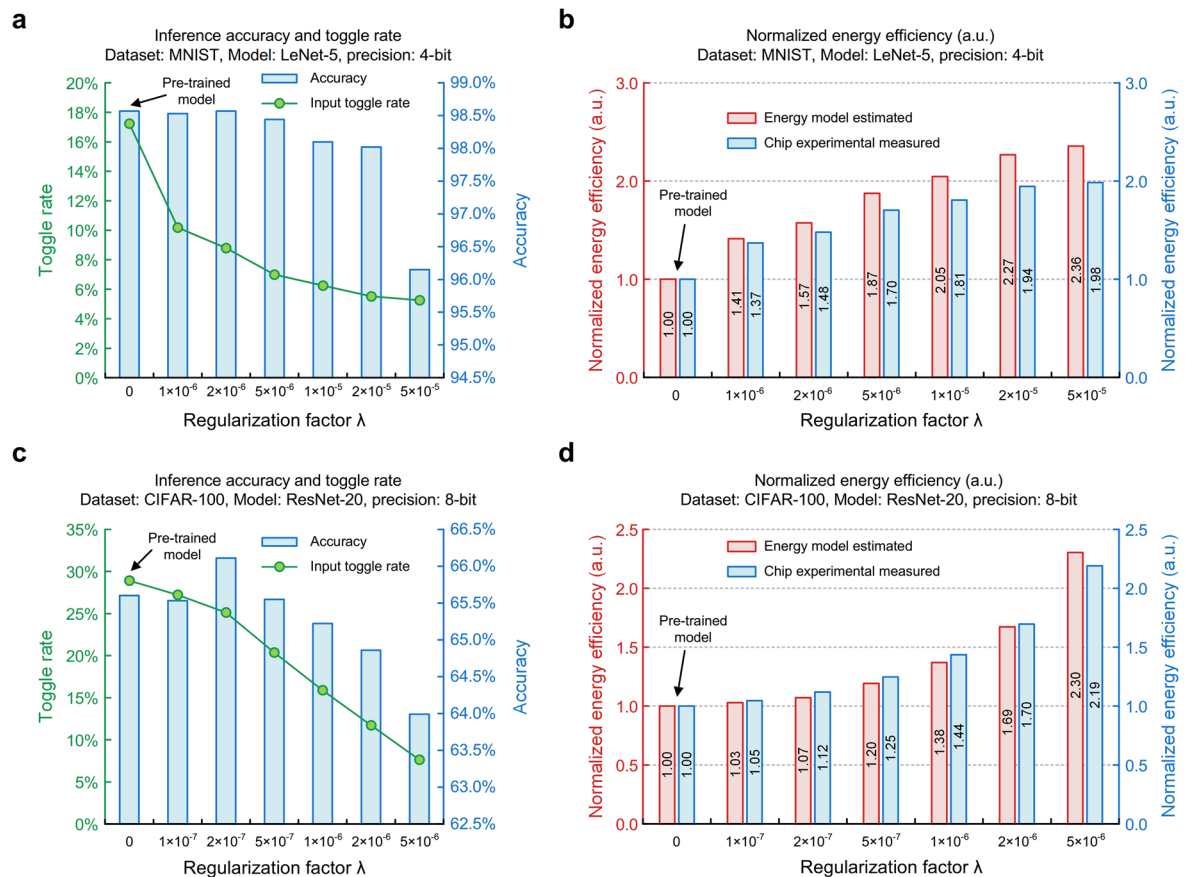
Extended Data Fig. 6 | PINN model quantization and performance. Flow field reconstruction with PINN: A comparison of predictions for u (streamwise velocity), v (spanwise velocity), and p (pressure) at varying computational precision levels, with computational fluid dynamics (CFD) data benchmark³². (a) CFD benchmark data. (b) Predictions from the FP32 PINN model. (c) Predictions from the INT16 PINN model. (d) Predictions from the INT12

PINN model. (e) Predictions from the INT8 PINN model. (f) Predictions from the INT4 PINN model. (g) Relative L_2 Norm (R_{L_2}): Temporal R_{L_2} for u with different bit precisions and the overall R_{L_2} for u . (h) Temporal R_{L_2} for v with different bit precisions and the overall R_{L_2} for v . (i) Temporal R_{L_2} for p with different bit precisions and the overall R_{L_2} for p .



Extended Data Fig. 7 | Energy model construction. In neural network workloads, convolution layers are mapped onto the nvCIM hardware to perform efficient MVM operation, with feature maps and kernels assigned to input drivers and memory banks. Dynamic energy consumption is calculated by

aggregating the energy costs of IBMD-bitcells, input drivers, and adders based on precomputed energy look-up tables. This model enables an evaluation of both model accuracy and energy consumption across convolutional and fully connected layers.



Extended Data Fig. 8 | Toggle-rate-aware training results. (a) Relationship between input toggle rate and accuracy for LeNet-5 INT4 model (dataset: MNIST). Increasing the regularization factor λ reduces the toggle rate while slightly impacting accuracy, demonstrating a tradeoff. (b) Energy efficiency improvement for LeNet-5. Reduction in toggle rate leads to notable energy efficiency gains, as illustrated by energy model estimations and chip-level

measurements. (c) Relationship between input toggle rate and accuracy for ResNet-20 INT8 model (dataset: CIFAR-100). Similar to LeNet-5, increasing λ reduces the toggle rate with minimal accuracy degradation. (d) Energy efficiency improvement for ResNet-20 (dataset: CIFAR-100). Larger λ values result in more energy efficiency improvements.

Extended Data Table 1 | Chip summary

Chip Summary		
Technology	40 nm CMOS	
Device	Foundry STT-MRAM	
Bitcell area (μm^2)	2.587	
MTJ critical diameter (nm)	78	
MTJ parameter	$R_p = 3.4 \text{ k}\Omega$, TMR = 170% (@ 0.1 V)	
Macro capacity	64 kb (1 kb per bank)	
Input precision (IN)	4b, 8b, 12b, 16b	
Weight precision (W)	4b, 8b, 12b, 16b	
Output precision (OUT)	16b, 20b, 24b, 28b, 32b, 36b, 40b	
Input channel	256	
Output channel	64 / 32 / 21 / 16	
Supply voltage (V)	0.65–1.20	
MVM computing frequency (MHz) @ 25°C	80–540	
Chip area (mm^2)	9.92 (including pads and test modules)	
Macro area (mm^2)	2.1	
Area efficiency (TOPS mm^{-2})	2.15	
MVM computing latency (ns) @VDD = 1.20 V	7.4	4bIN-4bW-16bOUT
	14.8	8bIN-8bW-24bOUT
	22.2	12bIN-12bW-32bOUT
	29.6	16bIN-16bW-40bOUT
MVM throughput (TOPS) @VDD = 1.20 V	4.42	4bIN-4bW-16bOUT
	1.11	8bIN-8bW-24bOUT
	0.49	12bIN-12bW-32bOUT
	0.28	16bIN-16bW-40bOUT
MVM energy efficiency (TOPS W^{-1}) @VDD = 0.65 V	112.3	4bIN-4bW-16bOUT
	28.1	8bIN-8bW-24bOUT
	12.5	12bIN-12bW-32bOUT
	7.02	16bIN-16bW-40bOUT

This table summarizes the key specifications of our nvDCIM macro, fabricated in a 40 nm CMOS process with foundry STT-MRAM. The macro supports multiple data precisions for input, weight, and output, and achieves a peak matrix–vector multiplication (MVM) frequency of 540 MHz at 25°C. Performance metrics such as MVM latency, throughput, and energy efficiency are reported across different precision settings and supply voltages.

Extended Data Table 2 | Comparison of nvDCIM with other nvCIM chips

	This work	Nat. El. '23 [22]	Nat. El. '23 [41]	Nature '22 [15]	Nature '22 [18]	Nat. El. '21 [12]	Nat. El. '21 [13]
Technology	40 nm	14 nm	22 nm	130 nm	28 nm	22 nm	22 nm
MAC type	Digital CIM	Analog CIM	Digital near-memory computing	Analog CIM	Analog CIM	Analog CIM	Analog CIM
Memory type	MRAM	PCM	MRAM	RRAM	MRAM	RRAM	RRAM
Bitcell size (μm^2)	2.587	4.094	N/A	1.69	0.933	0.026	0.026
Macro area (mm^2)	2.06	1.392	18	1.21	0.022*	N/A	N/A
Chip area (mm^2)	9.92	144	18	159	11.76*	6	6
Macro capacity	64 kb	64 kb	281.6 kb	64 kb	4 kb	512 kb	256 kb
Chip capacity	64 kb	4 Mb	6.6 Mb	3 Mb	4 kb	4 Mb	2 Mb
Input bit precision	4/8/12/16	8	1/8	1/2/4/8	1	1/2/4/8	1/2/4
Weight bit precision	4/8/12/16	3/4	2/4/8	4	1	2/4/8	2/4
Output bit precision	16/20/24/28/32/36/40	8	12/22/26	3/5/6/10	4	4/8/10/14	6/10/11
Lossless computation	Yes	No	Yes	No	No	No	No
Fully parallel computation	Yes	Yes	No	Yes	Yes	No	No
Non-volatile	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Peak frequency (MHz)	540	1000	360	0.74	11.1	97.1	54.6
MAC latency (ns)	7.4 (4b/4b)	520 (8b/analog)	22.2 (8b/8b)	1400 (1b/4b)	90.1 (1b/1b)	10.3 (4b/4b)	18.3 (4b/4b)
Peak throughput (TOPS)**	4.42 (@1.2 V) (4b/4b)	16.1 (@0.95 V) (8b/4b)	0.87 (@0.8 V) (8b/8b)	2.135 (@1.8 V) (1b/4b)	0.091 (@1.0 V) (1b/1b)	0.097 (@0.8 V) (4b/4b)	0.014 (@0.8 V) (4b/4b)
Normalized throughput (GOPS kb^{-1})***	69.1	3.93	0.13	0.695	22.7	0.023	0.007
Peak energy efficiency (TOPS W^{-1})	112.3 (@0.65 V) (4b/4b)	2.48 (@0.6 V) (8b/4b)	30.1 (@0.8 V) (8b/8b)	43 (@1.8 V) (1b/4b)	405 (@1.0 V) (1b/1b)	60.97 (@0.8 V) (4b/4b)	36.61 (@0.8 V) (4b/4b)
Normalized energy efficiency (TOPS $\text{W}^{-1} \text{ bit}$)****	1797	312	1926	172	405	976	586
Area efficiency (TOPS mm^{-2})	2.15	0.4	0.048	0.013	4.2	0.017	0.002

* Estimated from layout and chip photo.

** One multiply-accumulate operation = Two operations.

*** Normalized throughput = Peak throughput / Chip capacity.

**** Normalized energy efficiency = Energy efficiency $\times$ Input bit precision $\times$ Weight bit precision.

Comparison of key specifications between our nvDCIM chip and previously reported nvCIM chips. The proposed design achieves high energy efficiency and throughput while supporting fully parallel and lossless digital computation with flexible precisions.